# EE221 – Digital Logic Design

## Chapter 1

Dr. Hussain Ali

Department of Electrical Engineering
Military College of Signals, NUST

Fall 2025

# Table of Content


1. Different Number Systems
   - Decimal Number System
   - Octal Number System
   - Binary Number System
   - Hexadecimal Number System
2. Numbers with Different Bases
3. Number-Base Conversions
   - Decimal to Binary Conversion
   - Decimal to Octal Conversion
   - Decimal to Hexadecimal Conversion
4. Complements of Numbers
   - Complements of Binary Numbers
   - Subtraction with 1's Complement
   - Subtraction with 2's Complement
5. Signed Binary Numbers
   - Signed Magnitude & Signed Complement
6. Binary Codes: Binary-Coded Decimals

# Decimal Number System

4692

# Decimal Number System

4692.89

# Different Number Systems

$$
\begin{aligned}
4692.89 &= 4000 + 600 + 90 + 2 + 0.8 + 0.09 \\
&= 4 \times 1000 + 6 \times 100 + 9 \times 10 + 2 \times 1 + 8 \times 0.1 + 9 \times 0.01 \\
&= a_3 \times 10^3 + a_2 \times 10^2 + a_1 \times 10^1 + a_0 \times 10^0 + a_{-1} \times 10^{-1} + a_{-2} \times 10^{-2} \\
&= a_3 \times r^3 + a_2 \times r^2 + a_1 \times r^1 + a_0 \times r^0 + a_{-1} \times r^{-1} + a_{-2} \times r^{-2}
\end{aligned}
$$

where $a_n$'s are the coefficients:
$a_3 = 4, a_2 = 6, a_1 = 9, a_0 = 2, a_{-1} = 8, a_{-2} = 9$, and $r = 10$
In general:

$$\cdots + a_3 \times r^3 + a_2 \times r^2 + a_1 \times r^1 + a_0 \times r^0 + a_{-1} \times r^{-1} + a_{-2} \times r^{-2} + \cdots$$

- $r$ is called the *radix* or *base*
- $a_n$'s are the coefficients, ranging from 0 to $r - 1$
- For decimal: $r = 10$, $a_n \in \{0, 1, \ldots, 9\}$

# Decimal Number System – Counting

- How do we count in a decimal number system?
  - 0, 1, 2, 3, 4, 5, 6, 7, 8, 9
- What comes after 9? We have run out of numbers...

# Decimal Number System – Counting

- How do we count in a decimal number system?
  - 0, 1, 2, 3, 4, 5, 6, 7, 8, 9
- What comes after 9? We have run out of numbers...
- Remember units, tens, hundreds, thousands, etc.?
- We write 1 at the position of tens and a 0 at the position of units i.e., 10 and then continue increasing the units until 19

# Decimal Number System – Counting

- How do we count in a decimal number system?
  - 0, 1, 2, 3, 4, 5, 6, 7, 8, 9
- What comes after 9? We have run out of numbers...
- Remember units, tens, hundreds, thousands, etc.?
- We write 1 at the position of tens and a 0 at the position of units i.e., 10 and then continue increasing the units until 19
- Once at 19 we change the number at tens position and then start changing the number at units position

# Decimal Number System – Counting

- How do we count in a decimal number system?
  - 0, 1, 2, 3, 4, 5, 6, 7, 8, 9
- What comes after 9? We have run out of numbers...
- Remember units, tens, hundreds, thousands, etc.?
- We write 1 at the position of tens and a 0 at the position of units i.e., 10 and then continue increasing the units until 19
- Once at 19 we change the number at tens position and then start changing the number at units position
- When we reach 99, we put a 1 at hundreds position and continue
- and the counting goes on...

# Table of Content


1. Different Number Systems
   - Decimal Number System
   - Octal Number System
   - Binary Number System
   - Hexadecimal Number System
2. Numbers with Different Bases
3. Number-Base Conversions
   - Decimal to Binary Conversion
   - Decimal to Octal Conversion
   - Decimal to Hexadecimal Conversion
4. Complements of Numbers
   - Complements of Binary Numbers
   - Subtraction with 1's Complement
   - Subtraction with 2's Complement
5. Signed Binary Numbers
   - Signed Magnitude & Signed Complement
6. Binary Codes: Binary-Coded Decimals

- If the radix $r = 8$, we get the Octal number system
- Since $r = 8$, the coefficients $a_n$'s range from 0 to 7
- Valid octal numbers: 1234, 470, 2000, 1101, 777
- Invalid octal numbers: 811, 7901, 2838

- Counting in Octal:
  - 0, 1, 2, 3, 4, 5, 6, 7, 10, 11, 12, 13, 14, 15, 16, 17, 20, 21, 22, 23, ...
  - 65, 66, 67, 70, 71, 72, 73, 74, 75, 76, 77, 100, 101, 102, 103, 104, 105, 106, 107, 110, ...
- These are called *octal numbers*

- Example: Octal number 107

$$a_2 \times r^2 + a_1 \times r^1 + a_0 \times r^0 = 1 \times 8^2 + 0 \times 8^1 + 7 \times 8^0 = 71$$

- $(107)_8 = (71)_{10}$
- Try: $(1000)_8 = (512)_{10}$, $(999)_8$ is not an octal number

# Different Number Systems

## Octal Number System – with radix point

General form:

$$\cdots + a_3 \times 8^3 + a_2 \times 8^2 + a_1 \times 8^1 + a_0 \times 8^0 + a_{-1} \times 8^{-1} + a_{-2} \times 8^{-2} + \cdots$$

Example: $(107.73)_8$

$$\begin{aligned} &= 1 \times 8^2 + 0 \times 8^1 + 7 \times 8^0 + 7 \times 8^{-1} + 3 \times 8^{-2} \\ &= 64 + 0 + 7 + 0.875 + 0.046875 \\ &= (71.921875)_{10} \end{aligned}$$

# Table of Content


1. Different Number Systems
   - Decimal Number System
   - Octal Number System
   - Binary Number System
   - Hexadecimal Number System
2. Numbers with Different Bases
3. Number-Base Conversions
   - Decimal to Binary Conversion
   - Decimal to Octal Conversion
   - Decimal to Hexadecimal Conversion
4. Complements of Numbers
   - Complements of Binary Numbers
   - Subtraction with 1's Complement
   - Subtraction with 2's Complement
5. Signed Binary Numbers
   - Signed Magnitude & Signed Complement
6. Binary Codes: Binary-Coded Decimals

# Binary Number System

- Radix $r = 2$
- Coefficients $a_n \in \{0, 1\}$ (bits)
- Valid: 1111, 110, 1, 1101, 100
- Invalid: 123, 1120, 2010
- Decimal equivalent:

$$\cdots + a_3 \times 2^3 + a_2 \times 2^2 + a_1 \times 2^1 + a_0 \times 2^0$$

- Example: $(11)_2 = 1 \times 2^1 + 1 \times 2^0 = 2 + 1 = (3)_{10}$
- Example: $(101)_2 = 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = 4 + 1 = (5)_{10}$

- Example: $(1011)_2 = 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 8 + 2 + 1 = (11)_{10}$
- Example: $(101101)_2$ — use simpler method

# Binary Number System – with radix point

- Example: $(1011.11)_2$

$$= 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-1} + 1 \times 2^{-2}$$
$$= 8 + 2 + 1 + 0.5 + 0.25 = (11.75)_{10}$$

- Summary:
  - Binary: $r = 2$, bits: 0, 1
  - Octal: $r = 8$, digits: 0–7
  - Decimal: $r = 10$, digits: 0–9
- Another common system: $r = 16$ (Hexadecimal)

# Table of Content


1. Different Number Systems
   - Decimal Number System
   - Octal Number System
   - Binary Number System
   - Hexadecimal Number System
2. Numbers with Different Bases
3. Number-Base Conversions
   - Decimal to Binary Conversion
   - Decimal to Octal Conversion
   - Decimal to Hexadecimal Conversion
4. Complements of Numbers
   - Complements of Binary Numbers
   - Subtraction with 1's Complement
   - Subtraction with 2's Complement
5. Signed Binary Numbers
   - Signed Magnitude & Signed Complement
6. Binary Codes: Binary-Coded Decimals

# Hexadecimal Number System

- Radix $r = 16$
- Digits: 0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F
- Examples: 1E43, 1018, 2AB5D, FF0044, 5A4B3CDEF

# Hexadecimal Number System – Examples

- Example: $(A30C)_{16}$

$$
\begin{aligned}
&= A \times 16^3 + 3 \times 16^2 + 0 \times 16^1 + C \times 16^0 \\
&= 10 \times 4096 + 3 \times 256 + 0 \times 16 + 12 \times 1 \\
&= 41740 \\
&= (41740)_{10}
\end{aligned}
$$

# Table of Content


1. Different Number Systems
   - Decimal Number System
   - Octal Number System
   - Binary Number System
   - Hexadecimal Number System
2. Numbers with Different Bases
3. Number-Base Conversions
   - Decimal to Binary Conversion
   - Decimal to Octal Conversion
   - Decimal to Hexadecimal Conversion
4. Complements of Numbers
   - Complements of Binary Numbers
   - Subtraction with 1's Complement
   - Subtraction with 2's Complement
5. Signed Binary Numbers
   - Signed Magnitude & Signed Complement
6. Binary Codes: Binary-Coded Decimals

# Numbers with Different Bases – Summary

| Decimal (base 10) | Binary (base 2) | Octal (base 8) | Hexadecimal (base 16) |
|---|---|---|---|
| 00 | 0000 | 00 | 0 |
| 01 | 0001 | 01 | 1 |
| 02 | 0010 | 02 | 2 |
| 03 | 0011 | 03 | 3 |
| 04 | 0100 | 04 | 4 |
| 05 | 0101 | 05 | 5 |
| 06 | 0110 | 06 | 6 |
| 07 | 0111 | 07 | 7 |
| 08 | 1000 | 10 | 8 |
| 09 | 1001 | 11 | 9 |
| 10 | 1010 | 12 | A |
| 11 | 1011 | 13 | B |
| 12 | 1100 | 14 | C |
| 13 | 1101 | 15 | D |
| 14 | 1110 | 16 | E |
| 15 | 1111 | 17 | F |

**Binary to Octal Conversion**

Group binary digits into sets of three (from right to left for integer part, left to right for fractional part)

Example: $(101110011)_2$
$101\ 110\ 011 = 5\ 6\ 3 = (563)_8$

**Octal to Binary Conversion**

Convert each octal digit to its 3-bit binary equivalent

Example: $(347)_8$
$3 = 011$, $4 = 100$, $7 = 111 = (011100111)_2 = (11100111)_2$

**Binary to Hexadecimal Conversion**

Group binary digits into sets of four (from right to left for integer part, left to right for fractional part)

Example: $(1101011011)_2$
$11\ 0101\ 1011 = 3\ 5\ B = (35B)_{16}$

**Hexadecimal to Binary Conversion**

Convert each hexadecimal digit to its 4-bit binary equivalent

Example: $(A2F)_{16}$
$A = 1010$, $2 = 0010$, $F = 1111 = (101000101111)_2$

**Binary Arithmetic**

| Operation | Example | | Result |
|---|---|---|---|
| Addition | $1010_2$ | $(10_{10})$ | |
| | $+0110_2$ | $(+6_{10})$ | |
| | $10000_2$ | $(= 16_{10})$ | |
| Subtraction | $1010_2$ | $(10_{10})$ | |
| | $-0110_2$ | $(-6_{10})$ | |
| | $0100_2$ | $(= 4_{10})$ | |
| Multiplication | $1010_2$ | $(10_{10})$ | |
| | $\times 0011_2$ | $(\times 3_{10})$ | |
| | $11110_2$ | $(= 30_{10})$ | |

**Verification Examples**

- $(2)_{10} + (4)_{10} = (0010)_2 + (0100)_2 = (0110)_2 = (6)_{10}$
- $(6)_{10} + (4)_{10} = (0110)_2 + (0100)_2 = (1010)_2 = (10)_{10}$
- $(7)_{10} + (1)_{10} = (0111)_2 + (0001)_2 = (1000)_2 = (8)_{10}$
- $(2)_{10} + (4)_{10} = (2)_8 + (4)_8 = (6)_8 = (6)_{10}$
- $(5)_{10} + (7)_{10} = (5)_8 + (7)_8 = (14)_8 = (12)_{10}$
- $(9)_{10} + (6)_{10} = (11)_8 + (6)_8 = (17)_8 = (15)_{10}$

# Table of Content


1. Different Number Systems
   - Decimal Number System
   - Octal Number System
   - Binary Number System
   - Hexadecimal Number System
2. Numbers with Different Bases
3. Number-Base Conversions
   - Decimal to Binary Conversion
   - Decimal to Octal Conversion
   - Decimal to Hexadecimal Conversion
4. Complements of Numbers
   - Complements of Binary Numbers
   - Subtraction with 1's Complement
   - Subtraction with 2's Complement
5. Signed Binary Numbers
   - Signed Magnitude & Signed Complement
6. Binary Codes: Binary-Coded Decimals

**Conversion Methods**

- Binary to Octal and Octal to Binary
- Binary to Hexadecimal and Hexadecimal to Binary
- Decimal to Binary
- Decimal to Octal
- Decimal to Hexadecimal

**Decimal to Binary Conversion Algorithm**

1. Divide the decimal number by 2
2. Note down the quotient and the remainder
3. The remainder will be either 0 or 1 (this is $a_0$)
4. If the quotient is not 0, divide it by 2 and note down the quotient and remainder
5. The remainder will be either 0 or 1 (this is $a_1$)
6. Repeat the same process until the quotient becomes 0

**Example: Convert $(39)_{10}$ to binary**

| Divide by | Quotient | Remainder | Coefficient |
|---|---|---|---|
| 2 | 39 | | |
| 2 | 19 | 1 | $a_0 = 1$ |
| 2 | 9 | 1 | $a_1 = 1$ |
| 2 | 4 | 1 | $a_2 = 1$ |
| 2 | 2 | 0 | $a_3 = 0$ |
| 2 | 1 | 0 | $a_4 = 0$ |
| 2 | 0 | 1 | $a_5 = 1$ |

$$(39)_{10} = (a_5 a_4 a_3 a_2 a_1 a_0)_2 = (100111)_2$$

**Decimal Fraction to Binary Conversion**

1. Multiply the decimal fraction by 2
2. Note the integer part (will be 0 or 1)
3. Take the fractional part and repeat step 1
4. Continue until fractional part becomes 0 or desired precision is achieved

Example: Convert $(0.6875)_{10}$ to binary
$0.6875 \times 2 = 1.375 \rightarrow 1$
$0.375 \times 2 = 0.75 \rightarrow 0$
$0.75 \times 2 = 1.5 \rightarrow 1$
$0.5 \times 2 = 1.0 \rightarrow 1$
So, $(0.6875)_{10} = (0.1011)_2$

**Decimal to Octal Conversion Algorithm**

1. Divide the decimal number by 8
2. Note down the quotient and the remainder
3. If the quotient is not 0, divide it by 8 and note down the quotient and remainder
4. Repeat the same process until the quotient becomes 0

**Example: Convert $(39)_{10}$ to octal**

| Divide by | Quotient | Remainder | Coefficient |
|---|---|---|---|
| 8 | 39 | | |
| 8 | 4 | 7 | $a_0 = 7$ |
| 8 | 0 | 4 | $a_1 = 4$ |

$$(39)_{10} = (a_1 a_0)_8 = (47)_8$$

**Decimal Fraction to Octal Conversion**

1. Multiply the decimal fraction by 8
2. Note the integer part (0 to 7)
3. Take the fractional part and repeat step 1
4. Continue until fractional part becomes 0 or desired precision is achieved

Example: Convert $(0.513)_{10}$ to octal (approx.)
$0.513 \times 8 = 4.104 \rightarrow 4$
$0.104 \times 8 = 0.832 \rightarrow 0$
$0.832 \times 8 = 6.656 \rightarrow 6$
$0.656 \times 8 = 5.248 \rightarrow 5$
So, $(0.513)_{10} \approx (0.4065)_8$

**Decimal to Hexadecimal Conversion Algorithm**

1. Divide the decimal number by 16
2. Note down the quotient and the remainder
3. If the quotient is not 0, divide it by 16 and note down the quotient and remainder
4. Repeat the same process until the quotient becomes 0

**Example: Convert $(39)_{10}$ to hexadecimal**

| Divide by | Quotient | Remainder | Coefficient |
|---|---|---|---|
| 16 | 39 | | |
| 16 | 2 | 7 | $a_0 = 7$ |
| 16 | 0 | 2 | $a_1 = 2$ |

$$(39)_{10} = (a_1 a_0)_{16} = (27)_{16}$$

**Example: Convert $(1738)_{10}$ to hexadecimal**

| Divide by | Quotient | Remainder | Coefficient |
|---|---|---|---|
| 16 | 1738 | | |
| 16 | 108 | 10 | $a_0 = A$ |
| 16 | 6 | 12 | $a_1 = C$ |
| 16 | 0 | 6 | $a_2 = 6$ |

$$(1738)_{10} = (a_2 a_1 a_0)_{16} = (6CA)_{16}$$

# Table of Content


1. Different Number Systems
   - Decimal Number System
   - Octal Number System
   - Binary Number System
   - Hexadecimal Number System
2. Numbers with Different Bases
3. Number-Base Conversions
   - Decimal to Binary Conversion
   - Decimal to Octal Conversion
   - Decimal to Hexadecimal Conversion
4. Complements of Numbers
   - Complements of Binary Numbers
   - Subtraction with 1's Complement
   - Subtraction with 2's Complement
5. Signed Binary Numbers
   - Signed Magnitude & Signed Complement
6. Binary Codes: Binary-Coded Decimals

# Complements of Numbers

- **Complements are used to simplify subtraction operation in digital computers**
- **Simplification of operation has multiple advantages:**
  - It results in simpler circuits (convenience in designing process)
  - It results in low cost (simpler circuit means fewer and simpler hardware components)

**Types of Complements**

- Diminished Radix Complement
- Radix Complement

# Complements of Binary Numbers

## 1's Complement (Diminished Radix Complement)

- The 1's complement of a binary number is obtained by subtracting each bit from 1
- There are two possibilities:
  - If the bit is 0 then its 1's complement is $1 - 0 = 1$
  - If the bit is 1 then its 1's complement is $1 - 1 = 0$
- Therefore, we simply flip each bit to find the 1's complement of a binary number

**Example:**

| | |
|---|---|
| Consider a binary number: | 1011001 |
| The 1's complement is: | 0100110 |

# Complements of Binary Numbers

## 2's Complement (Radix Complement)

- The 2's complement of a binary number is obtained by adding 1 to the 1's complement of the number
- It can be obtained directly by:
  - Starting from the right, copy bits until the first 1
  - Flip the remaining bits

**Examples:**

| | |
|---|---|
| Consider a binary number: | 1011000 |
| The 2's complement is: | 0101000 |
| Consider a binary number: | 1001001 |
| The 2's complement is: | 0110111 |

# Complements of Numbers

## Complements of Binary Numbers

- The complement of the complement restores the number to its original value
- $X = \text{Complement}(\text{Complement}(X))$
- This is true for both 1's and 2's complements

**Examples:**

| | | |
|---|---|---|
| | Consider a binary number: | 1011001 |
| 1's complement Example: | The 1's complement is: | 0100110 |
| | The 1's complement of the complement is: | 1011001 |
| | Consider a binary number: | 1001001 |
| 2's complement Example: | The 2's complement is: | 0110111 |
| | The 2's complement of the complement is: | 1001001 |

# Subtraction with 1's Complement

**Steps for subtraction using 1's complement:**

1. Take the 1's complement of the subtrahend (Y)
2. Add this to the minuend (X)
3. If there is an end carry, add 1 to the result
4. If there is no end carry, take the 1's complement of the result and add a negative sign

**Example:** $X = 11101, Y = 10111$

$$
\begin{aligned}
X &= 11101 \\
\text{1's complement of } Y &= 01000 \\
\text{Sum} &= 100101 \\
\text{End carry} + 1 & \\
X - Y &= 00110
\end{aligned}
$$

# Subtraction with 2's Complement

**Steps for subtraction using 2's complement:**

1. Take the 2's complement of the subtrahend (Y)
2. Add this to the minuend (X)
3. If there is an end carry, discard it
4. If there is no end carry, take the 2's complement of the result and add a negative sign

**Example:** $X = 11101, Y = 10111$

$$X = 11101$$
$$\text{2's complement of } Y = 01001$$
$$\text{Sum} = 100110$$
$$\text{Discard End carry} \checkmark$$
$$X - Y = 00110$$

# Subtraction with Complements

## When $X < Y$ (1's complement)

**Example:** $X = 10111, Y = 11101$

$$
\begin{aligned}
X &= 10111 \\
\text{1's complement of } Y &= 00010 \\
\text{Sum} &= 11001 \\
\text{1's complement of the sum} &= 00110 \\
X - Y &= -00110
\end{aligned}
$$

# Subtraction with Complements

## When $X < Y$ (2's complement)

**Example:** $X = 10111, Y = 11101$

$$X = 10111$$
$$\text{2's complement of } Y = 00011$$
$$\text{Sum} = 11010$$
$$\text{2's complement of the sum} = 00110$$
$$X - Y = -00110$$

# Table of Content


1. Different Number Systems
   - Decimal Number System
   - Octal Number System
   - Binary Number System
   - Hexadecimal Number System
2. Numbers with Different Bases
3. Number-Base Conversions
   - Decimal to Binary Conversion
   - Decimal to Octal Conversion
   - Decimal to Hexadecimal Conversion
4. Complements of Numbers
   - Complements of Binary Numbers
   - Subtraction with 1's Complement
   - Subtraction with 2's Complement
5. Signed Binary Numbers
   - Signed Magnitude & Signed Complement
6. Binary Codes: Binary-Coded Decimals

# Signed Binary Numbers – Signed Magnitude

| Decimal | Signed Magnitude |
|---|---|
| +7 | 0111 |
| +6 | 0110 |
| +5 | 0101 |
| +4 | 0100 |
| +3 | 0011 |
| +2 | 0010 |
| +1 | 0001 |
| +0 | 0000 |
| -0 | 1000 |
| -1 | 1001 |
| -2 | 1010 |
| -3 | 1011 |
| -4 | 1100 |
| -5 | 1101 |
| -6 | 1110 |
| -7 | 1111 |
| -8 | — |

- In computers, everything must be represented by bits
- It is customary to represent the sign with a bit placed in the left-most position of a number
- The convention is to use sign bit $= 0$ for positive numbers, and sign bit $= 1$ for negative numbers
- We call this notation signed-magnitude
- The table shows 4-bits signed binary numbers

# Signed Binary Numbers – Signed Magnitude

| Decimal | 5-bit Signed Magnitude | Decimal | 5-bit Signed Magnitude |
|---|---|---|---|
| +15 | 01111 | -0 | 10000 |
| +14 | 01110 | -1 | 10001 |
| +13 | 01101 | -2 | 10010 |
| +12 | 01100 | -3 | 10011 |
| +11 | 01011 | -4 | 10100 |
| +10 | 01010 | -5 | 10101 |
| +9 | 01001 | -6 | 10110 |
| +8 | 01000 | -7 | 10111 |
| +7 | 00111 | -8 | 11000 |
| +6 | 00110 | -9 | 11001 |
| +5 | 00101 | -10 | 11010 |
| +4 | 00100 | -11 | 11011 |
| +3 | 00011 | -12 | 11100 |
| +2 | 00010 | -13 | 11101 |
| +1 | 00001 | -14 | 11110 |
| +0 | 00000 | -15 | 11111 |

# Signed Binary Numbers – Signed Magnitude

**Examples: What decimal numbers do the following signed-magnitude numbers represent?**

- $X = 1001 \rightarrow$ -1 (4-bit)
- $X = 0110 \rightarrow +6$ (4-bit)
- $X = 00001111 \rightarrow +15$ (8-bit)
- $X = 10000011 \rightarrow$ -3 (8-bit)
- $X = 11001111 \rightarrow$ -79 (8-bit)
- $X = 110111 \rightarrow$ -23 (6-bit)

Signed-magnitude notation cannot be used in arithmetic operations directly.
Try to solve: $(2)_{10} - (3)_{10}$, $(5)_{10} - (4)_{10}$

# Signed Binary Numbers – Signed Complement

| Decimal | Signed-2's Complement | Signed-1's Complement |
|---|---|---|
| +7 | 0111 | 0111 |
| +6 | 0110 | 0110 |
| +5 | 0101 | 0101 |
| +4 | 0100 | 0100 |
| +3 | 0011 | 0011 |
| +2 | 0010 | 0010 |
| +1 | 0001 | 0001 |
| +0 | 0000 | 0000 |
| -0 | – | 1111 |
| -1 | 1111 | 1110 |
| -2 | 1110 | 1101 |
| -3 | 1101 | 1100 |
| -4 | 1100 | 1011 |
| -5 | 1011 | 1010 |
| -6 | 1010 | 1001 |
| -7 | 1001 | 1000 |
| -8 | 1000 | – |

- 1's and 2's complement can also be used to represent signed numbers
- Since usage of 2's complement simplifies computations, we almost always use it to represent negative numbers
- As the leftmost bit of a positive number will always be 0, the leftmost bit of its 2's complement will always be 1
- The presence of this 1 denotes that it is a negative number
- We call this notation signed-complement

# Signed Binary Numbers

## Signed Binary Numbers in Signed 2's-Complement System

- Here are some examples of 8-bit signed binary numbers
- The leftmost bit is reserved for signs. Remaining 7 bits are used for numbers
- How many numbers can be represented using 8 bits? $2^8 = 256$

**Examples: Positive signed numbers:**

- +62: 00111110
- +99: 01100011

**Examples: Negative signed numbers:**

- -62: 11000010
- -99: 10011101

**Example:** $+99 - 62 = +99 + (-62)$

$$
\begin{aligned}
+99 &= 01100011 \\
-62 &= 11000010 \\
\text{Sum} &= 100100101 \quad \text{(discard overflow bit)} \\
&= 00100101 = 37_{10}
\end{aligned}
$$

The answer should be 37.

# Table of Content


1. Different Number Systems
   - Decimal Number System
   - Octal Number System
   - Binary Number System
   - Hexadecimal Number System
2. Numbers with Different Bases
3. Number-Base Conversions
   - Decimal to Binary Conversion
   - Decimal to Octal Conversion
   - Decimal to Hexadecimal Conversion
4. Complements of Numbers
   - Complements of Binary Numbers
   - Subtraction with 1's Complement
   - Subtraction with 2's Complement
5. Signed Binary Numbers
   - Signed Magnitude & Signed Complement
6. Binary Codes: Binary-Coded Decimals

# Binary Codes

- Digital devices use signals that have two states $\rightarrow$ binary
- Information processed by these devices is in the form of binary numbers $\rightarrow$ 0's and 1's
- We as humans interact with digital devices using decimal numbers, alphabets, etc.
- When we press a key, a binary code is generated and sent to the digital circuit
- Each key is encoded $\rightarrow$ for each key there is a unique binary sequence i.e., a number of bits
- For example: for the key of 1 on keyboard the binary sequence might be 00000010 and for the key of A it might be 01100010

# Binary Codes

## A very simple example

**The 10 decimal digits can be encoded using 10 bits as follows:**

| Decimal Digit | 10-bit Code |
|---|---|
| 0 | 0000000001 |
| 1 | 0000000010 |
| 2 | 0000000100 |
| 3 | 0000001000 |
| 4 | 0000010000 |
| 5 | 0000100000 |
| 6 | 0001000000 |
| 7 | 0010000000 |
| 8 | 0100000000 |
| 9 | 1000000000 |

Example: $(128)_{10}$ will be encoded as:
0000000010 0000000100 0100000000 $\rightarrow$ 30 bits!

It is not a numeric code. Example: $1 + 2 \neq 3$

## BCD Encoding

| Decimal Symbol | BCD Digit |
|---|---|
| 0 | 0000 |
| 1 | 0001 |
| 2 | 0010 |
| 3 | 0011 |
| 4 | 0100 |
| 5 | 0101 |
| 6 | 0110 |
| 7 | 0111 |
| 8 | 1000 |
| 9 | 1001 |

- Each decimal number is represented by a binary code of 4-bits
- We call it binary-coded decimal (BCD)

# Binary Codes

## Binary-Coded Decimals (BCD)

- BCD is a way to express each decimal digit with a binary code
- It is very easy to convert between decimal and BCD – main advantage
- We like to read and write in decimal. Therefore, BCD provides an excellent way to communicate with binary systems (keypads, calculators, keyboards, keypad screens etc.)
- Four bits can be used to represent 16 numbers
- In BCD we utilize only ten. The remaining 6 code combinations are not used – invalid codes
- The invalid codes are: $1010, 1011, 1100, 1101, 1110, 1111$

# Binary Codes

## Binary-Coded Decimals – How to Express a Decimal Number in BCD

**Simply replace each decimal digit with the appropriate 4-bit code**
**Examples:**

- Convert 82 to BCD:

$$8 \rightarrow 1000$$
$$2 \rightarrow 0010$$
$$\text{BCD} = 1000\ 0010$$

- Convert 106 to BCD:

$$1 \rightarrow 0001$$
$$0 \rightarrow 0000$$
$$6 \rightarrow 0110$$
$$\text{BCD} = 0001\ 0000\ 0110$$

- Convert 9012 to BCD:

$$9 \rightarrow 1001$$
$$0 \rightarrow 0000$$
$$1 \rightarrow 0001$$
$$2 \rightarrow 0010$$
$$\text{BCD} = 1001\ 0000\ 0001\ 0010$$

# Binary Codes

## Binary-Coded Decimals – How to Determine a Decimal Number From a BCD

**Start from the right-most bit and break the code into group of four bits**
**Write the decimal digit represented by each 4-bits group**
**Examples:**

- Convert 10011001 to decimal:

$$1001 \rightarrow 9$$
$$1001 \rightarrow 9$$
$$\text{Decimal} = 99$$

- Convert 000100111001 to decimal:

$$0001 \rightarrow 1$$
$$0011 \rightarrow 3$$
$$1001 \rightarrow 9$$
$$\text{Decimal} = 139$$

**BCD Addition Steps:**

1. Add the two BCD numbers using the rules of binary addition
2. If a 4-bit sum is equal to or less than 9, it is a valid BCD number
3. If a 4-bit sum is greater than 9 OR if a carry out of the 4-bit group is generated, it is an invalid result. Add 6 (0110) to the 4-bit sum in order to skip the six invalid states and return to a valid BCD code. If a carry results when 6 is added, simply add the carry to the next 4-bit group

**Example:** Add 00010011 (13) + 00100110 (26)

$$\begin{array}{rl} 0001\ 0011 & (13) \\ +0010\ 0110 & (+26) \\ \hline 0011\ 1001 & (39) \end{array}$$

Valid BCD result: 39

**Example:** Add 1001 (9) + 0100 (4)

| | |
|---|---|
| 1001 | (9) |
| +0100 | (+4) |
| 1101 | (Invalid BCD number $> 9$) |
| +0110 | (Add 6) |
| 0001 0011 | (13) |

Valid BCD result: 13

**Example:** Add 1001 (9) + 1001 (9)

| | |
|---|---|
| 1001 | (9) |
| +1001 | (+9) |
| 1 0010 | (Invalid because of carry) |
| +0110 | (Add 6 to LSD) |
| 0001 1000 | (18) |

Valid BCD result: 18

**Example:** Add 00010110 (16) + 00010101 (15)

| | |
|---|---|
| 0001 0110 | (16) |
| +0001 0101 | (+15) |
| 0010 1011 | (Right group invalid $> 9$) |
| +0000 0110 | (Add 6 to invalid code) |
| 0011 0001 | (31) |

Valid BCD result: 31

# Other Decimal Codes

| Decimal Digit | BCD 8421 | 2421 | Excess-3 | 8, 4, -2, -1 |
|---|---|---|---|---|
| 0 | 0000 | 0000 | 0011 | 0000 |
| 1 | 0001 | 0001 | 0100 | 0111 |
| 2 | 0010 | 0010 | 0101 | 0110 |
| 3 | 0011 | 0011 | 0110 | 0101 |
| 4 | 0100 | 0100 | 0111 | 0100 |
| 5 | 0101 | 1011 | 1000 | 1011 |
| 6 | 0110 | 1100 | 1001 | 1010 |
| 7 | 0111 | 1101 | 1010 | 1001 |
| 8 | 1000 | 1110 | 1011 | 1000 |
| 9 | 1001 | 1111 | 1100 | 1111 |
| | Unused bit combinations | | | |
| | 1010 | 0101 | 0000 | 0001 |
| | 1011 | 0110 | 0001 | 0010 |
| | 1100 | 0111 | 0010 | 0011 |
| | 1101 | 1000 | 1101 | 1100 |
| | 1110 | 1001 | 1110 | 1110 |
| | 1111 | 1010 | 1111 | 1111 |

# Weighted Codes

- 8421, 2421, and (8,4,-2,-1) are weighted codes
- Excess-3 is an unweighted code
- 2421 and Excess-3 codes exhibit self-complementing property
- Self-complementing: 9's complement of a decimal number is obtained directly by flipping the bits of the Excess-3 code of that decimal number

# Excess-3 Code

- **Excess-3 is an unweighted code**
- In **Excess-3** each coded combination is obtained from the corresponding binary value plus 3
- **Excess-3 code exhibits self-complementing property**
- Self-complementing: 9's complement of a decimal number is obtained directly by flipping the bits of the **Excess-3 code** of that decimal number

**Example:** Consider a decimal number: 428
Using **Excess-3 code** the decimal number 428 is represented as: 0111 0101 1011
9's complement of 428 is: 999-428 = 571
9's complement of 428 in encoded form (Excess-3 form) is: 1000 1010 0100 (flip each bit)
This corresponds to the 1's complement of a binary number

2421 code is also a self-complementing code

# Gray Code

| Decimal | Binary | Gray |
|---|---|---|
| 0 | 0000 | 0000 |
| 1 | 0001 | 0001 |
| 2 | 0010 | 0011 |
| 3 | 0011 | 0010 |
| 4 | 0100 | 0110 |
| 5 | 0101 | 0111 |
| 6 | 0110 | 0101 |
| 7 | 0111 | 0100 |
| 8 | 1000 | 1100 |
| 9 | 1001 | 1101 |
| 10 | 1010 | 1111 |
| 11 | 1011 | 1110 |
| 12 | 1100 | 1010 |
| 13 | 1101 | 1011 |
| 14 | 1110 | 1001 |
| 15 | 1111 | 1000 |

- Unweighted code, Non-numeric code
- Only a single bit changes from one code word to the next in sequence
- No two codes are identical, Gray codes can have any number of bits

# ASCII Character Code

- Standard code for alphanumeric characters
- 7-bit code (a byte is usually used, additional codes used to represent Greek or italic type fonts)
- Example: The code for A is 1000001
  The code for a is 1100001

| Control Character | Description | Control Character | Description |
|---|---|---|---|
| NUL | Null | DLE | Data-link escape |
| SOH | Start of heading | DC1 | Device control 1 |
| STX | Start of text | DC2 | Device control 2 |
| ETX | End of text | DC3 | Device control 3 |
| EOT | End of transmission | DC4 | Device control 4 |
| ENQ | Enquiry | NAK | Negative acknowledge |
| ACK | Acknowledge | SYN | Synchronous idle |
| BEL | Bell | ETB | End-of-transmission block |
| BS | Backspace | CAN | Cancel |
| HT | Horizontal tab | EM | End of medium |
| LF | Line feed | SUB | Substitute |
| VT | Vertical tab | ESC | Escape |
| FF | Form feed | FS | File separator |
| CR | Carriage return | GS | Group separator |
| SO | Shift out | RS | Record separator |
| SI | Shift in | US | Unit separator |
| SP | Space | DEL | Delete |

# Error-Detecting Code

- A parity bit is added to the ASCII codes to detect errors
- A parity bit is an extra bit to make the total number of 1's either even or odd
- The code for **A** is 1000001 $\rightarrow$ number of 1's is 2
- Insert a parity bit on the left side to make the total number of 1's either even or odd
- The code for **A** with even parity is 01000001
- The code for **A** with odd parity is 11000001

# Table of Content


1. Different Number Systems
   - Decimal Number System
   - Octal Number System
   - Binary Number System
   - Hexadecimal Number System
2. Numbers with Different Bases
3. Number-Base Conversions
   - Decimal to Binary Conversion
   - Decimal to Octal Conversion
   - Decimal to Hexadecimal Conversion
4. Complements of Numbers
   - Complements of Binary Numbers
   - Subtraction with 1's Complement
   - Subtraction with 2's Complement
5. Signed Binary Numbers
   - Signed Magnitude & Signed Complement
6. Binary Codes: Binary-Coded Decimals

# Thank You!

Questions?

# EE221 – Digital Logic Design

## Chapter 2 – Boolean Algebra and Logic Gates

Dr. Hussain Ali
Department of Electrical Engineering
Military College of Signals
National University of Sciences and Technology, Islamabad

Military College of Signals

Fall 2025, Dr Hussain Ali, MCS, NUST
September 08, 2025

# Table of Content


1. Basic Theorems and Properties of Boolean Algebra
   - Postulates
   - Theorems
2. Boolean Functions
3. Truth Tables
4. Logic Circuits
5. Canonical and Standard Forms
6. Other Logic Operations
7. Digital Logic Gates
8. Applications

# Basic Theorems and Properties of Boolean Algebra

## Postulates of Boolean Algebra

| | | |
|---|---|---|
| Postulate 2 | (a) $x + 0 = x$ | (b) $x \cdot 1 = x$ |
| Postulate 3 (Comm.) | (a) $x + y = y + x$ | (b) $x \cdot y = y \cdot x$ |
| Postulate 4 (Dist.) | (a) $x \cdot (y + z) = x \cdot y + x \cdot z$ | (b) $x + (y \cdot z) = (x + y) \cdot (x + z)$ |
| Postulate 5 | (a) $x + x' = 1$ | (b) $x \cdot x' = 0$ |

# Basic Theorems and Properties of Boolean Algebra

## Postulates of Boolean Algebra

| | | |
|---|---|---|
| Postulate 2 | (a) $x + 0 = x$ | (b) $x \cdot 1 = x$ |
| Postulate 3 (Comm.) | (a) $x + y = y + x$ | (b) $x \cdot y = y \cdot x$ |
| Postulate 4 (Dist.) | (a) $x \cdot (y + z) = x \cdot y + x \cdot z$ | (b) $x + (y \cdot z) = (x + y) \cdot (x + z)$ |
| Postulate 5 | (a) $x + x' = 1$ | (b) $x \cdot x' = 0$ |

Let's verify some of these postulates using truth tables

# Basic Theorems and Properties of Boolean Algebra

## Theorems of Boolean Algebra

| | | |
|---|---|---|
| Theorem 1 | (a) $x + x = x$ | (b) $x \cdot x = x$ |
| Theorem 2 | (a) $x + 1 = 1$ | (b) $x \cdot 0 = 0$ |
| Theorem 3 (involution) | $(x')' = x$ | |
| Theorem 4 (associative) | (a) $x + (y + z) = (x + y) + z$ | (b) $x \cdot (y \cdot z) = (x \cdot y)$ |
| Theorem 5 (DeMorgan) | (a) $(x + y)' = x' \cdot y'$ | (b) $(x \cdot y)' = x' + y'$ |
| Theorem 6 (absorption) | (a) $x + x \cdot y = x$ | (b) $x \cdot (x + y) = x$ |

Let's prove some of these theorems

**Operator Precedence**

1. Parentheses
2. NOT
3. AND
4. OR

- Boolean algebra deals with binary variables and logic operations
- Boolean function is an algebraic expression consisting of binary variables, the constants (0, and 1), and logic operation symbols
- A Boolean function can be equal to either 0 or 1
- Some examples of Boolean functions are given below:

**Examples:**

$$F_1 = x + y'z$$
$$F_2 = x'y'z + xyz + x'yz + xy'z$$
$$F_3 = x'yz + xz$$
$$F_4 = (x + y)'(x' + y')$$
$$F_5 = xy + x(wz + wz')$$
$$F_6 = (yz' + x'w)(xy' + zw')$$
$$F_7 = (x' + z')(x + y' + z')$$
$$F_8 = x + xz$$

Using the basic theorems and postulates of Boolean algebra, simplify the following Boolean functions:

$$
\begin{aligned}
F_2 &= x'y'z + xyz + x'yz + xy'z \\
&= x'z(y + y') + xz(y + y') \\
&= x'z + xz \\
&= (x' + x)z \\
&= z \\
F_3 &= x'yz + xz \\
&= (x'y + x)z \\
&= (x + x')(x + y)z \\
&= (x + y)z
\end{aligned}
$$

We will now simplify $F_4 = (x + y)'(x' + y')$

$$\begin{aligned} F_4 &= (x + y)'(x' + y') \\ &= x'y'(x' + y') \quad \text{DeMorgan's theorem 1} \\ &= x'y'x' + x'y'y' \\ &= x'y' + x'y' \\ &= x'y' \end{aligned}$$

**Reminder:** $(xy)' = x' + y'$ is DeMorgan's theorem 2

- A Boolean function can be represented in a truth table
- The number of rows in a truth table is $2^n$ where $n$ is the number of variables in the function/expression
- The binary combinations of the variables are obtained by counting from 0 to $2^n - 1$

# Basic Theorems and Properties of Boolean Algebra

## Truth Tables

- A Boolean function can be represented in a truth table
- The number of rows in a truth table is $2^n$ where $n$ is the number of variables in the function/expression
- The binary combinations of the variables are obtained by counting from 0 to $2^n - 1$
- Let's make the truth table for $F_1 = x + y'z$

**Example:**

| $x$ | $y$ | $z$ | $y'$ | $y'z$ | $F_1 = x + y'z$ |
|---|---|---|---|---|---|
| 0 | 0 | 0 | 1 | 0 | 0 |
| 0 | 0 | 1 | 1 | 1 | 1 |
| 0 | 1 | 0 | 0 | 0 | 0 |
| 0 | 1 | 1 | 0 | 0 | 0 |
| 1 | 0 | 0 | 1 | 0 | 1 |
| 1 | 0 | 1 | 1 | 1 | 1 |
| 1 | 1 | 0 | 0 | 0 | 1 |
| 1 | 1 | 1 | 0 | 0 | 1 |

As another example, let's select the expression $F_4 = (x + y)'(x' + y')$ **Example**

| $x$ | $y$ | $x + y$ | $(x + y)'$ | $x'$ | $y'$ | $x' + y'$ | $F_4$ |
|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 1 | 1 | 1 | 1 | 1 |
| 0 | 1 | 1 | 0 | 1 | 0 | 1 | 0 |
| 1 | 0 | 1 | 0 | 0 | 1 | 1 | 0 |
| 1 | 1 | 1 | 0 | 0 | 0 | 0 | 0 |

Let us now simplify the expression $F_4$

$$\begin{aligned} F_4 &= (x+y)'(x'+y') \\ &= x'y'(x'+y') \quad \text{using DeMorgan's theorem} \\ &= x'y'+x'y' = x'y' \end{aligned}$$

Let us now simplify the expression $F_4$

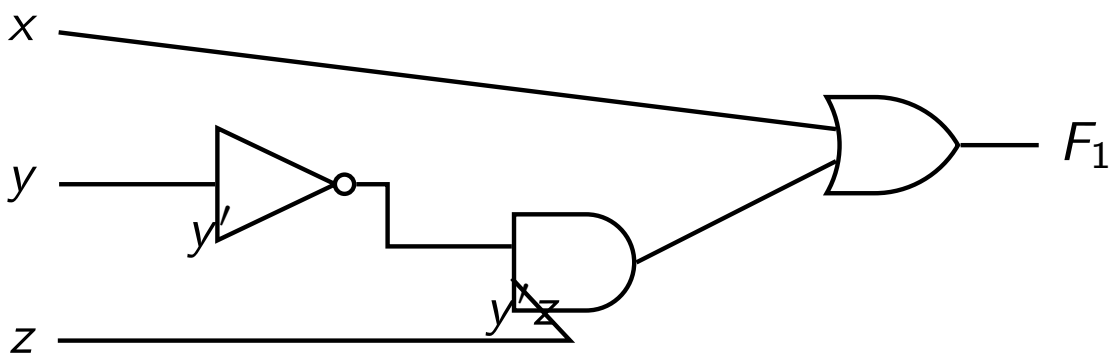
$$\begin{aligned} F_4 &= (x+y)'(x'+y') \\ &= x'y'(x'+y') \quad \text{using DeMorgan's theorem} \\ &= x'y'+x'y' = x'y' \end{aligned}$$

Let's see if the simplified version is indeed correct. To do that, we will make another truth table as shown below **Example**

| $x$ | $y$ | $x'$ | $y'$ | $x'y'$ |
|---|---|---|---|---|
| 0 | 0 | 1 | 1 | 1 |
| 0 | 1 | 1 | 0 | 0 |
| 1 | 0 | 0 | 1 | 0 |
| 1 | 1 | 0 | 0 | 0 |

# Logic Circuits

- A Boolean function can be transformed into a circuit diagram consisting of logic gates connected in a particular structure
- The logic circuit diagram is also called a schematic
- The logic circuit diagram for $F_1 = x + y'z$ is shown below:




- There is an inverter (NOT gate) for $y$ to generate its complement $y'$
- AND gate generates $y'z$
- OR gate generates $x + y'z$

# Building Logic Circuits After Manipulating Boolean Expressions

- We have already seen that a simpler expression can be found for a given Boolean function by using the rules of Boolean algebra
- This allows one to reduce the number of gates required to implement a function
- This might also result in reduction of the number of inputs
- Motivation for you engineers: reduce the cost of the circuit
- This is an important design task
- Let's see an example

Consider a Boolean function: $F_2 = x'y'z + x'yz + xy'$
This could be simplified to: $F_2 = x'z + xy'$
The two circuits (before and after simplification are shown below)

(a) $F_2 = x'y'z + x'yz + xy'$ (b) $F_2 = xy' + x'z$

# Basic Theorems and Properties of Boolean Algebra

## Some Observations

- If we have a Boolean function, then there is only one way for it to be represented as a truth table
- A Boolean function can be simplified or even modified to another expression
- The particular expression used to represent a Boolean function dictates the structure of its equivalent logic circuit – and, therefore, the number of gates in it
- Conversely, the interconnection of logic gates will determine the logic expression
- We can use the rules of Boolean algebra to simplify expressions and thus save cost!

Let's learn a trick to simplify expressions that look impossible to simplify further

**Example**

$$\begin{aligned} xy + x'z + yz &= xy + x'z + yz(x + x') \\ &= xy + x'z + xyz + x'yz \\ &= xy(1 + z) + x'z(1 + y) \\ &= xy + x'z \end{aligned}$$

# Some Tricks...

Some more tricks to simplify expressions quickly
Repeated application of DeMorgan's theorem

$$(x + y + z)' = x'y'z'$$

Complement of Functions
Take the dual of the function and complement each literal

$$F_1 = x'yz' + x'y'z$$
$$F_1' = ?$$
$$\text{dual of } F_1 = (x' + y + z')(x' + y' + z)$$
$$\text{complement of each literal} = (x + y' + z)(x + y + z') = F_1'$$

- Introduction

- Introduction
- Two-variables case

- Introduction
- Two-variables case
- Three-variable case

| x | y | z | Minterms | | Maxterms | |
|---|---|---|---|---|---|---|
| | | | Term | Designation | Term | Designation |
| 0 | 0 | 0 | $x'y'z'$ | $m_0$ | $x+y+z$ | $M_0$ |
| 0 | 0 | 1 | $x'y'z$ | $m_1$ | $x+y+z'$ | $M_1$ |
| 0 | 1 | 0 | $x'yz'$ | $m_2$ | $x+y'+z$ | $M_2$ |
| 0 | 1 | 1 | $x'yz$ | $m_3$ | $x+y'+z'$ | $M_3$ |
| 1 | 0 | 0 | $xy'z'$ | $m_4$ | $x'+y+z$ | $M_4$ |
| 1 | 0 | 1 | $xy'z$ | $m_5$ | $x'+y+z'$ | $M_5$ |
| 1 | 1 | 0 | $xyz'$ | $m_6$ | $x'+y'+z$ | $M_6$ |
| 1 | 1 | 1 | $xyz$ | $m_7$ | $x'+y'+z'$ | $M_7$ |

- Minterms are also called standard products
- Maxterms are also called standard sums

- Sum of Minterms (Sum = OR)
- Product of Maxterms (Product = AND)
- Conversion between Canonical Forms

| x | y | z | F |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 |
| 0 | 1 | 0 | 0 |
| 0 | 1 | 1 | 1 |
| 1 | 0 | 0 | 0 |
| 1 | 0 | 1 | 0 |
| 1 | 1 | 0 | 1 |
| 1 | 1 | 1 | 1 |

$$F = xy + x'z$$

$$F(x, y, z) = x'y'z + x'yz + xyz' + xyz = \Sigma(1, 3, 6, 7)$$

$$F(x, y, z) = (x + y + z)(x + y' + z)(x' + y + z)(x' + y + z') = \Pi(0, 2, 4, 5)$$

# Standard Forms

- In Canonical forms, functions have terms that contain all variables
- In Standard forms, terms of a function may contain one, two, or any number of literals/variables
- Sum of Products (SOP)
- Product of Sums (POS)

**Examples:**

SOP:

$$F_1 = x'yz + x'y' + xy + z$$
$$F_2 = x + y' + xyz'$$
$$F_3 = xyz$$

POS:

$$F_4 = (x' + y + z)(x' + y')(x + y + z)$$
$$F_5 = (x + y')(x + y + z')z$$

Non-standard:

$$F_6 = x'(y + z) + z(x'y' + x) + z$$

# Standard Forms

- Standard forms result in two-level implementation unlike non-standard form
- However, there could be a downside $\rightarrow$ number of inputs to a gate could be impractical
- We prefer two-level implementation whenever practical $\rightarrow$ least amount of delay

  (a) $AB + CD + E$ (b) $AB + CD + CE$

- To draw a two-level diagram, function must be written in a standard form (SOP or POS)

# Basic Theorems and Properties of Boolean Algebra

## Other Logic Operations

| x | y | F0 | F1 | F2 | F3 | F4 | F5 | F6 | F7 | F8 | F9 | F10 | F11 | F12 | F13 | F14 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 0 | 1 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 0 | 0 | 0 | 0 | 1 | 1 | 1 |
| 1 | 0 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 1 |
| 1 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 |

- Two functions that produce a constant 0 or 1.
- Four functions with unary operations: complement and transfer
- Ten functions with binary operators that define eight different operations: AND, OR, NAND, NOR, exclusive-OR, equivalence, inhibition, and implication

# Basic Theorems and Properties of Boolean Algebra

## Other Logic Operations

| Boolean Functions | Operator Symbol | Name | Comments |
|---|---|---|---|
| $F_0 = 0$ | | Null | Binary constant 0 |
| $F_1 = xy$ | $x \cdot y$ | AND | $x$ and $y$ |
| $F_2 = xy'$ | $x/y$ | Inhibition | $x$, but not $y$ |
| $F_3 = x$ | | Transfer | $x$ |
| $F_4 = x'y$ | $y/x$ | Inhibition | $y$, but not $x$ |
| $F_5 = y$ | | Transfer | $y$ |
| $F_6 = xy' + x'y$ | $x \oplus y$ | Exclusive-OR | $x$ or $y$, but not both |
| $F_7 = x + y$ | $x + y$ | OR | |
| $F_8 = (x + y)'$ | $x \downarrow y$ | NOR | Not-OR |
| $F_9 = xy + x'y'$ | $(x \oplus y)'$ | Equivalence | $x$ equals $y$ |
| $F_{10} = y'$ | $y'$ | Complement | Not $y$ |
| $F_{11} = x + y'$ | $x \subset y$ | Implication | If $y$, then $x$ |
| $F_{12} = x'$ | $x'$ | Complement | Not $x$ |
| $F_{13} = x' + y$ | $x \supset y$ | Implication | If $x$, then $y$ |
| $F_{14} = (xy)'$ | $x \uparrow y$ | NAND | Not-AND |
| $F_{15} = 1$ | | Identity | Binary constant 1 |

- NOR function is the complement of OR function (not-OR)
- NAND function is the complement of AND function (not-AND)
- XOR (exclusive OR) is similar to OR but excludes the combination of both x and y being equal to 1
- Equivalence is 1 when x and y are equal
- Equivalence is complement of XOR, therefore also called exclusive NOR (XNOR)

# Basic Theorems and Properties of Boolean Algebra

## Digital Logic Gates

- A manufacturing plant uses two tanks to store a liquid chemical that is used in a manufacturing process.
- Each tank has a sensor that detects when the chemical level drops to 25% of full.
- The sensors produce a 5 V signal (logic 1) when the tanks are more than 25% full and 0 V (logic 0) if less.
- Green light switches ON when both tanks are above the 25% level.
- Use logic gates to implement this function?
- Let's say we need the output of our logic circuit to be 1 if both tanks are above the 25% level.

# Basic Theorems and Properties of Boolean Algebra

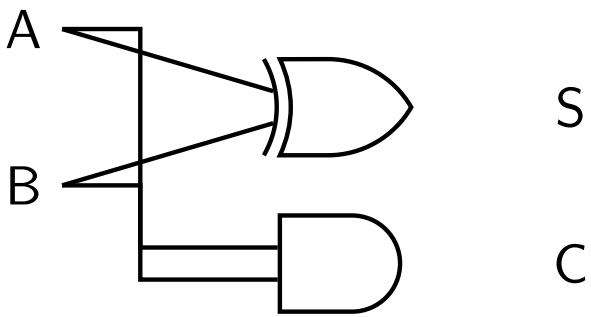
## Application I

- Consider the manufacturing plant of Application I.
- Now we need to display a red light when the chemical level in at least one of the tanks drops to 25% of full.
- The sensors produce a 5 V signal (logic 1) when the tanks are more than 25% full and 0 V (logic 0) if less.
- Use logic gates to implement this function?
- Let's say we need the output of our logic circuit to be 1 if at least one of the tanks drops to 25% of full.

# Basic Theorems and Properties of Boolean Algebra

## Application II

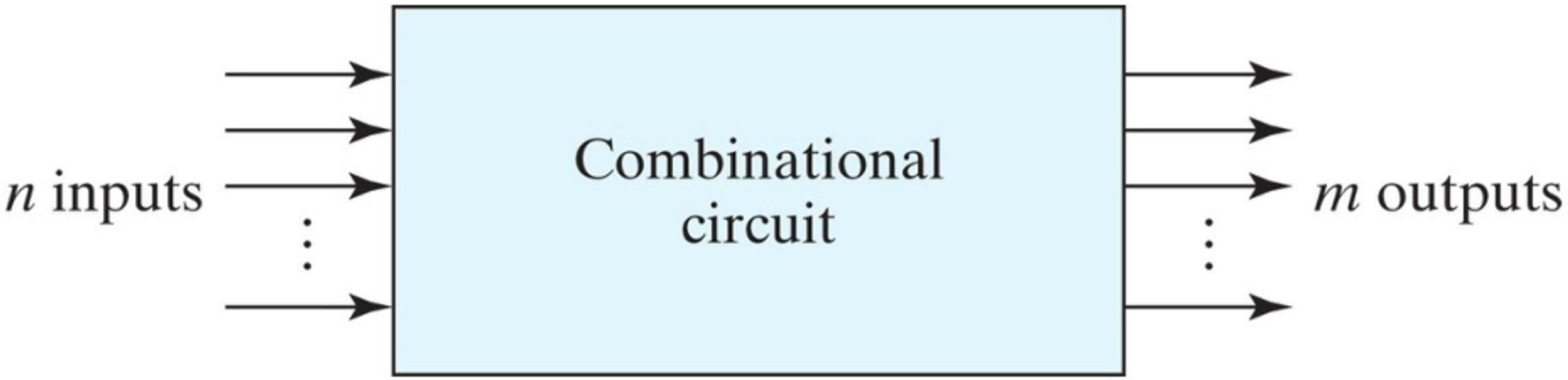
To add two bits, we can use a combination of a XOR and an AND gate as follows




| A | B | Carry | Sum |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 1 | 0 | 1 |
| 1 | 0 | 0 | 1 |
| 1 | 1 | 1 | 0 |

# End of Chapter 2

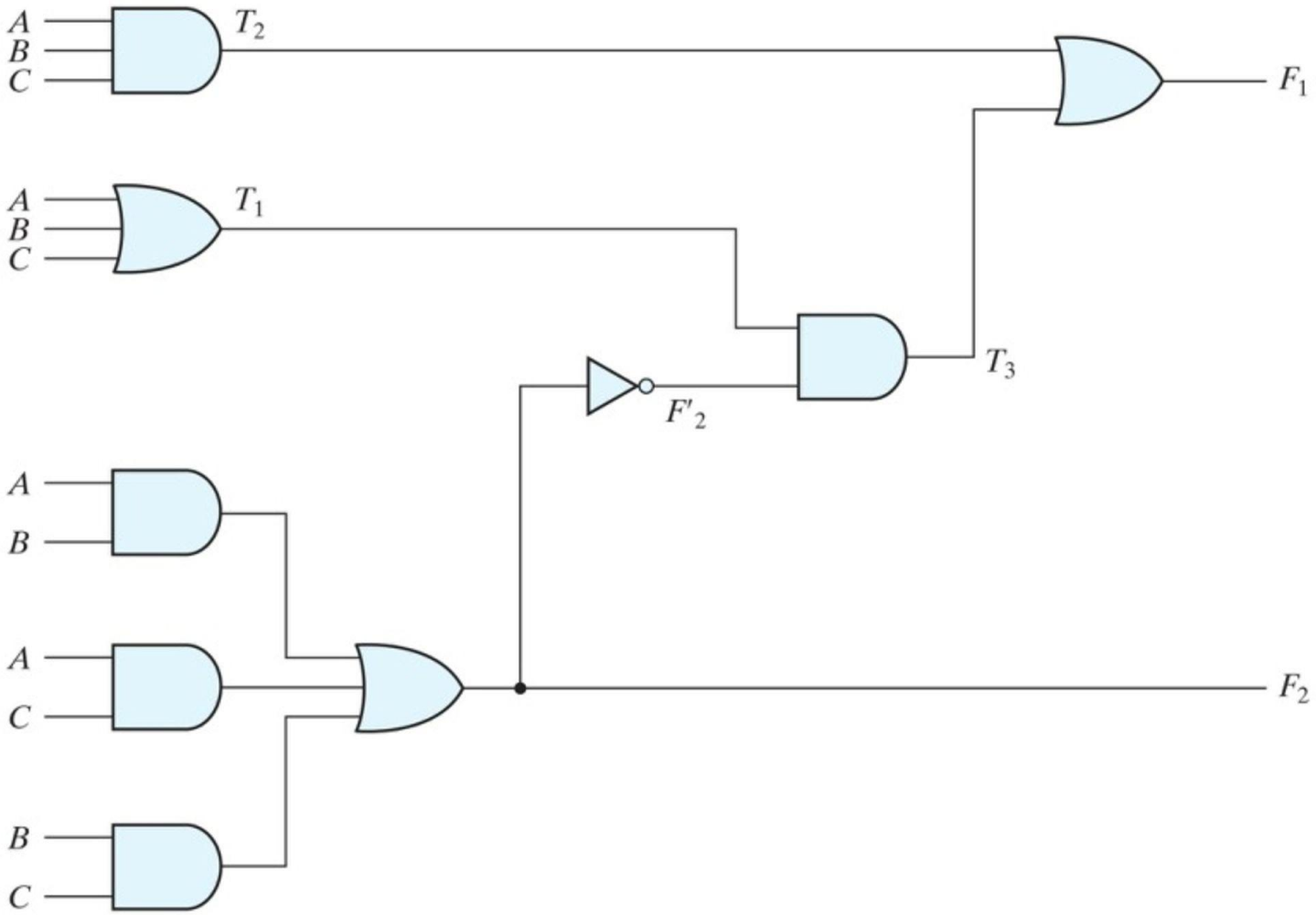
**FIGURE 4.1** Block diagram of combinational circuit




Copyright ©2013 Pearson Education, publishing as Prentice Hall


Copyright ©2013 by Pearson Education, Inc.
All rights reserved.

**FIGURE 4.2** Logic diagram for analysis example




Copyright ©2013 Pearson Education, publishing as Prentice Hall


Copyright ©2013 by Pearson Education, Inc.
All rights reserved.
PEARSON

**Table 4.1** Truth Table for the Logic Diagram of Fig. 4.2

## Table 4.1
### *Truth Table for the Logic Diagram of Fig. 4.2*

| $A$ | $B$ | $C$ | $F_2$ | $F'_2$ | $T_1$ | $T_2$ | $T_3$ | $F_1$ |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 | 1 | 1 | 0 | 1 | 1 |
| 0 | 1 | 0 | 0 | 1 | 1 | 0 | 1 | 1 |
| 0 | 1 | 1 | 1 | 0 | 1 | 0 | 0 | 0 |
| 1 | 0 | 0 | 0 | 1 | 1 | 0 | 1 | 1 |
| 1 | 0 | 1 | 1 | 0 | 1 | 0 | 0 | 0 |
| 1 | 1 | 0 | 1 | 0 | 1 | 0 | 0 | 0 |
| 1 | 1 | 1 | 1 | 0 | 1 | 1 | 0 | 1 |

Copyright ©2012 Pearson Education, publishing as Prentice Hall


Copyright ©2013 by Pearson Education, Inc.
All rights reserved.

**Table 4.2** Truth Table for Code Conversion Example

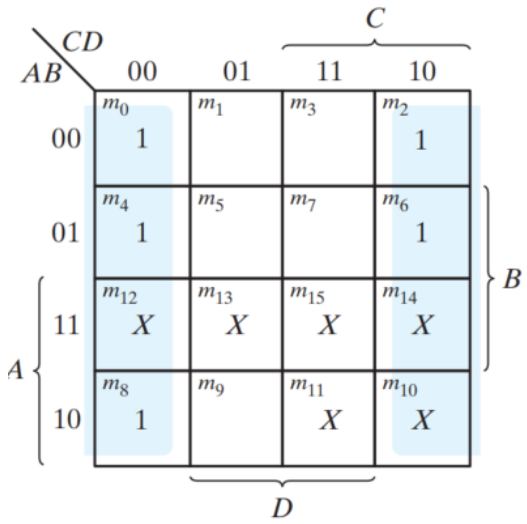
## Table 4.2
*Truth Table for Code Conversion Example*

| Input BCD | | | | Output Excess-3 Code | | | |
|---|---|---|---|---|---|---|---|
| *A* | *B* | *C* | *D* | *w* | *x* | *y* | *z* |
| 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1 |
| 0 | 0 | 0 | 1 | 0 | 1 | 0 | 0 |
| 0 | 0 | 1 | 0 | 0 | 1 | 0 | 1 |
| 0 | 0 | 1 | 1 | 0 | 1 | 1 | 0 |
| 0 | 1 | 0 | 0 | 0 | 1 | 1 | 1 |
| 0 | 1 | 0 | 1 | 1 | 0 | 0 | 0 |
| 0 | 1 | 1 | 0 | 1 | 0 | 0 | 1 |
| 0 | 1 | 1 | 1 | 1 | 0 | 1 | 0 |
| 1 | 0 | 0 | 0 | 1 | 0 | 1 | 1 |
| 1 | 0 | 0 | 1 | 1 | 1 | 0 | 0 |

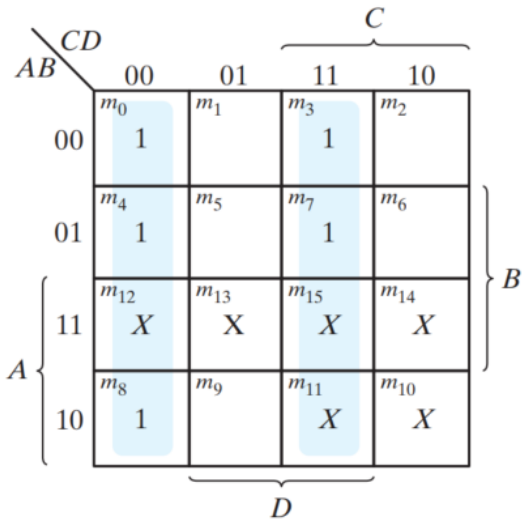
Copyright ©2012 Pearson Education, publishing as Prentice Hall


Copyright ©2013 by Pearson Education, Inc.
All rights reserved. PEARSON

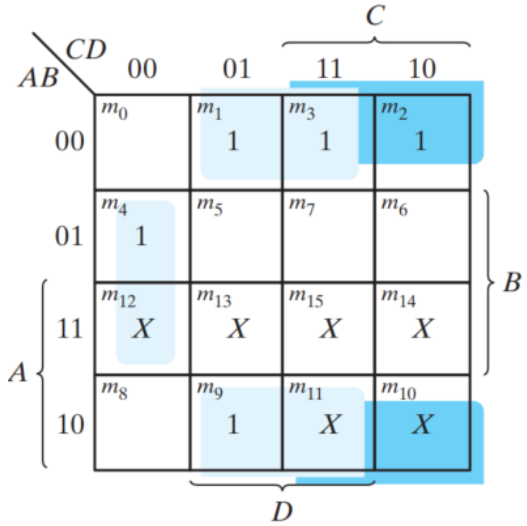
**FIGURE 4.3** Maps for BCD-to-excess-3 code converter



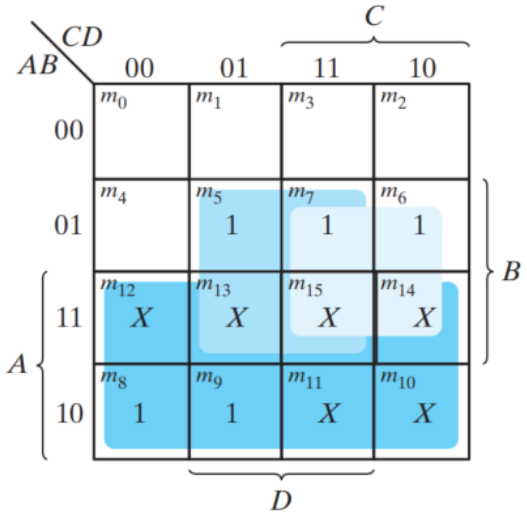

$z = D'$




$y = CD + C'D'$



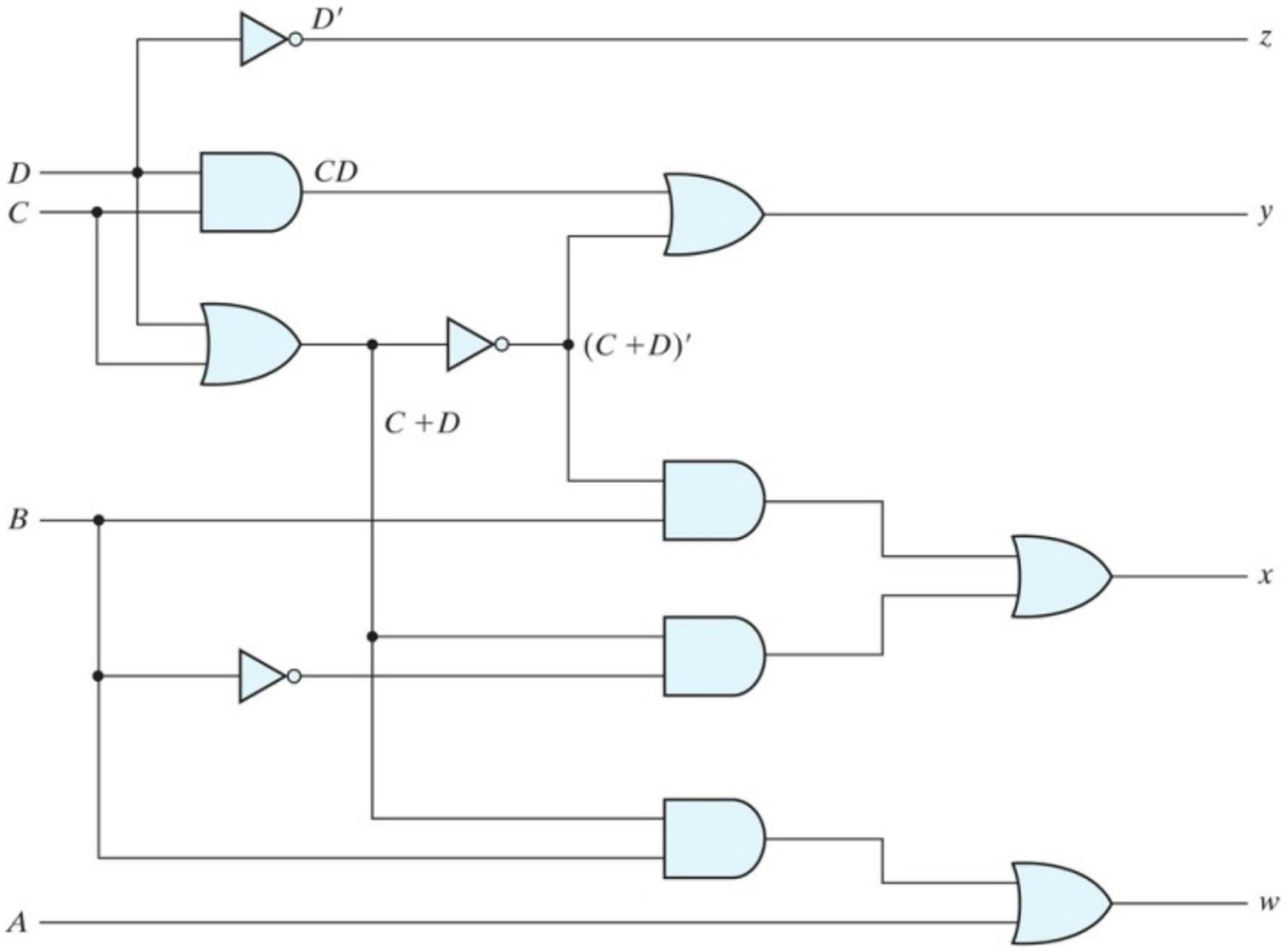

$x = B'C + B'D + BC'D'$




$w = A + BC + BD$


Copyright ©2013 by Pearson Education, Inc.
All rights reserved. PEARSON

**FIGURE 4.4** Logic diagram for BCD-to-excess-3 code converter




Copyright ©2013 Pearson Education, publishing as Prentice Hall


Copyright ©2013 by Pearson Education, Inc.
All rights reserved.
PEARSON

**Table 4.3** Half Adder

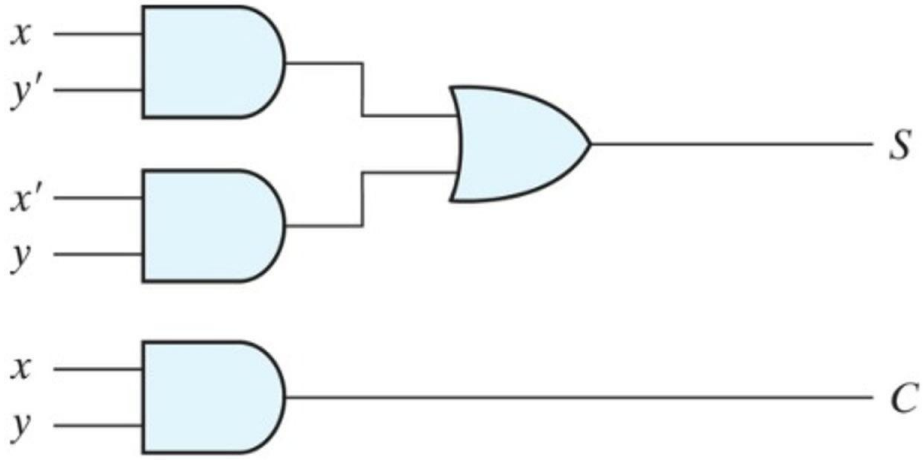
# Table 4.3
## *Half Adder*

| *x* | *y* | *C* | *S* |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 1 | 0 | 1 |
| 1 | 0 | 0 | 1 |
| 1 | 1 | 1 | 0 |

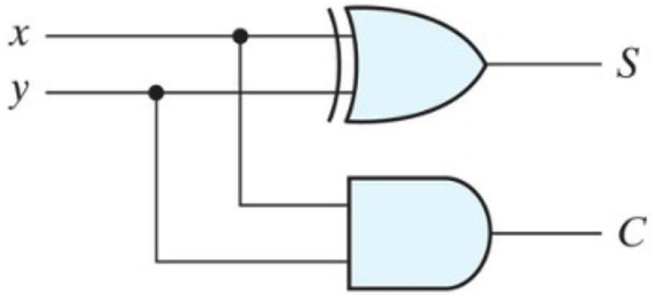
Copyright ©2012 Pearson Education, publishing as Prentice Hall


Copyright ©2013 by Pearson Education, Inc.
All rights reserved.

**FIGURE 4.5** Implementation of half adder




(a) $S = xy' + x'y$
$C = xy$




(b) $S = x \oplus y$
$C = xy$

Copyright ©2013 Pearson Education, publishing as Prentice Hall


Copyright ©2013 by Pearson Education, Inc.
All rights reserved. PEARSON

**Table 4.4** Full Adder

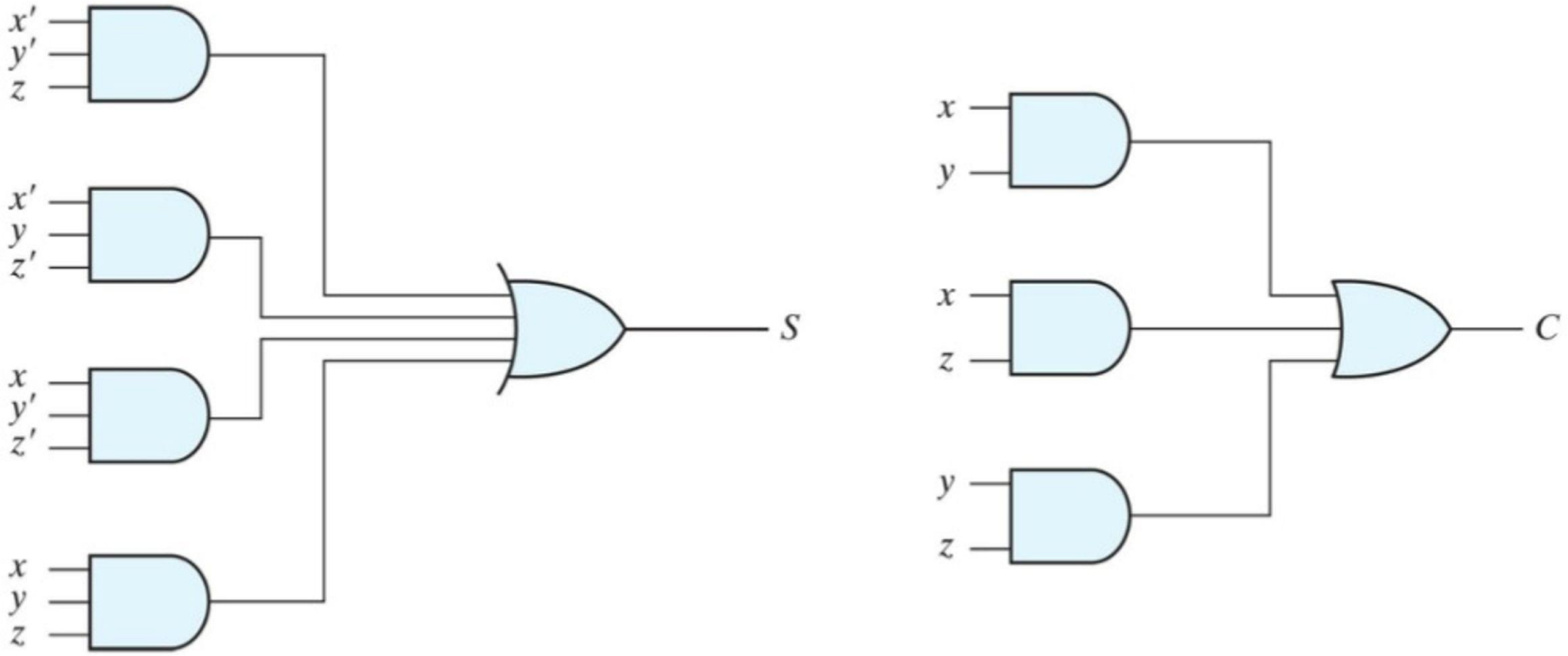
## Table 4.4
## *Full Adder*

| *x* | *y* | *z* | *C* | *S* |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 | 1 |
| 0 | 1 | 0 | 0 | 1 |
| 0 | 1 | 1 | 1 | 0 |
| 1 | 0 | 0 | 0 | 1 |
| 1 | 0 | 1 | 1 | 0 |
| 1 | 1 | 0 | 1 | 0 |
| 1 | 1 | 1 | 1 | 1 |

Copyright ©2012 Pearson Education, publishing as Prentice Hall


Copyright ©2013 by Pearson Education, Inc.
All rights reserved. PEARSON

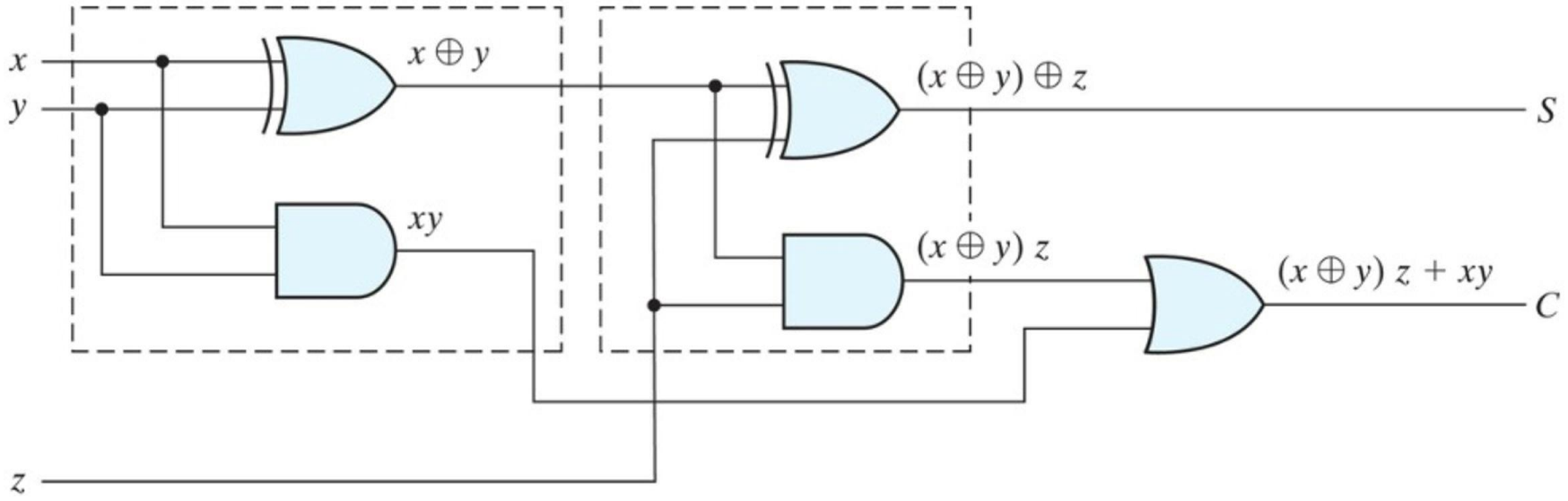
**FIGURE 4.7** Implementation of full adder in sum-of-products form




Copyright ©2013 Pearson Education, publishing as Prentice Hall


Copyright ©2013 by Pearson Education, Inc.
All rights reserved.
PEARSON

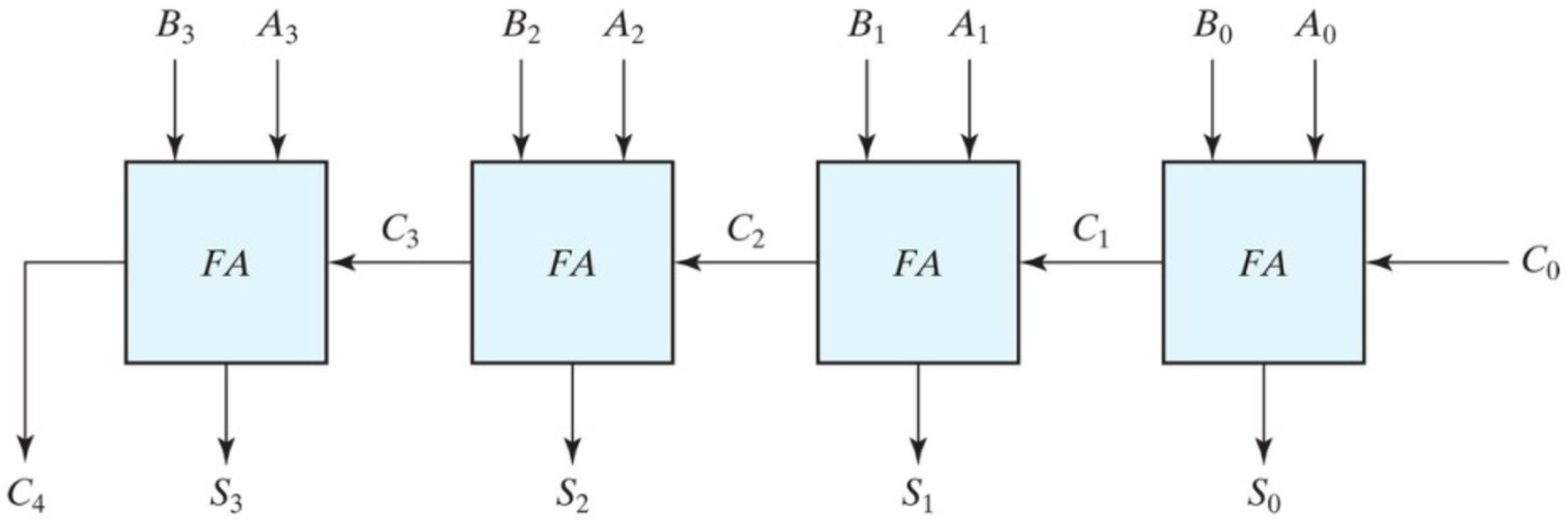
**FIGURE 4.8** Implementation of full adder with two half adders and an OR gate




Copyright ©2013 Pearson Education, publishing as Prentice Hall


Copyright ©2013 by Pearson Education, Inc.
All rights reserved.
PEARSON

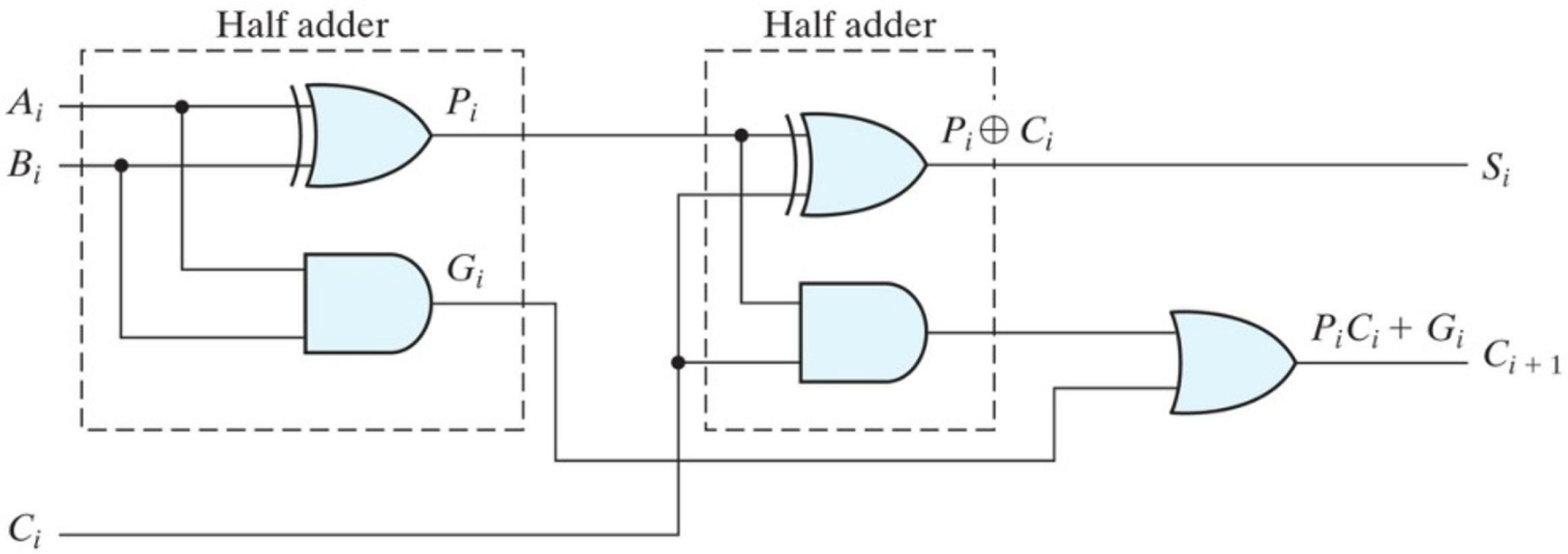
**FIGURE 4.9** Four-bit adder




Copyright ©2013 Pearson Education, publishing as Prentice Hall


Copyright ©2013 by Pearson Education, Inc.
All rights reserved. PEARSON

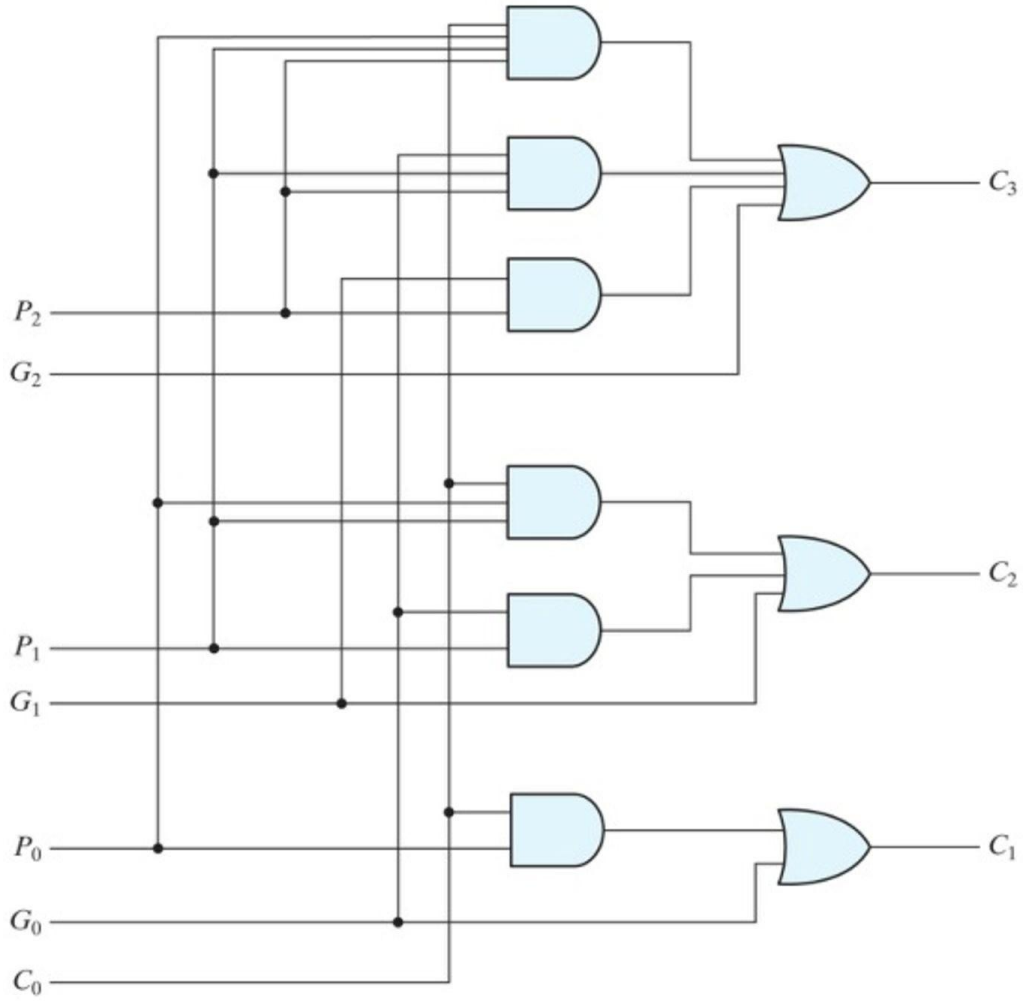
**FIGURE 4.10** Full adder with *P* and *G* shown




Copyright ©2013 Pearson Education, publishing as Prentice Hall


Copyright ©2013 by Pearson Education, Inc.
All rights reserved.
PEARSON

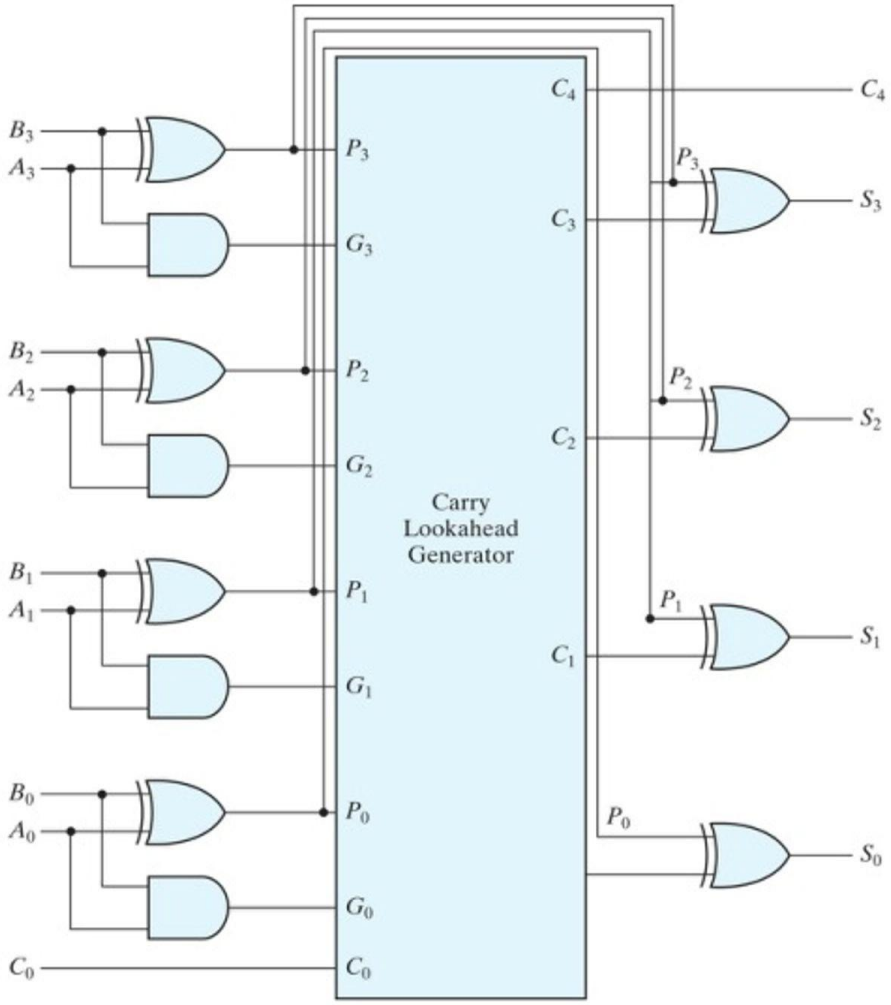
**FIGURE 4.11** Logic diagram of carry lookahead generator




Copyright ©2013 Pearson Education, publishing as Prentice Hall


Copyright ©2013 by Pearson Education, Inc.
All rights reserved. PEARSON

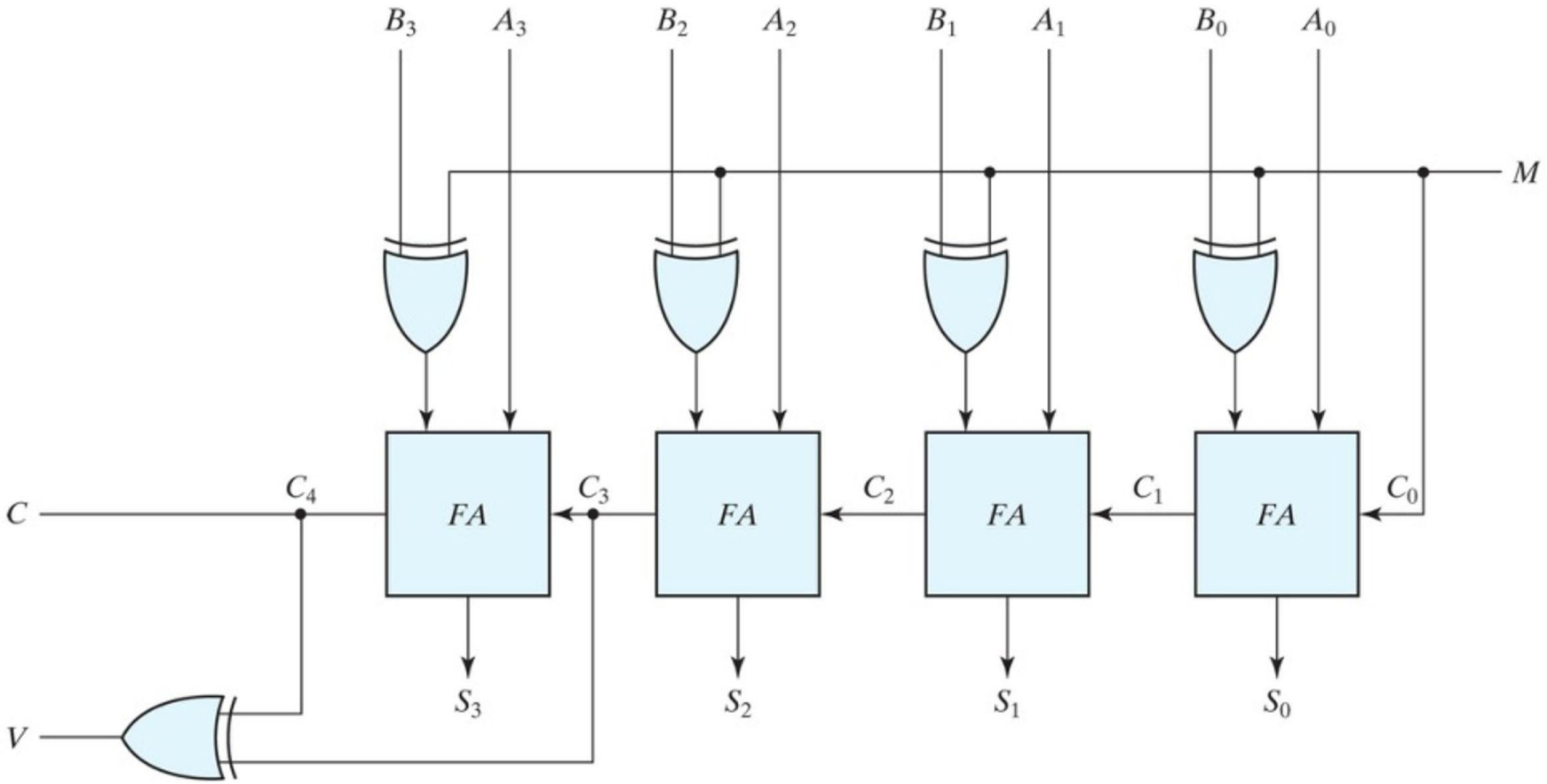
**FIGURE 4.12** Four-bit adder with carry lookahead




Copyright ©2013 Pearson Education, publishing as Prentice Hall


Copyright ©2013 by Pearson Education, Inc.
All rights reserved.
PEARSON

**FIGURE 4.13** Four-bit adder–subtractor (with overflow detection)




Copyright ©2013 Pearson Education, publishing as Prentice Hall


Copyright ©2013 by Pearson Education, Inc.
All rights reserved.
PEARSON

**Table 4.5** Derivation of BCD Adder

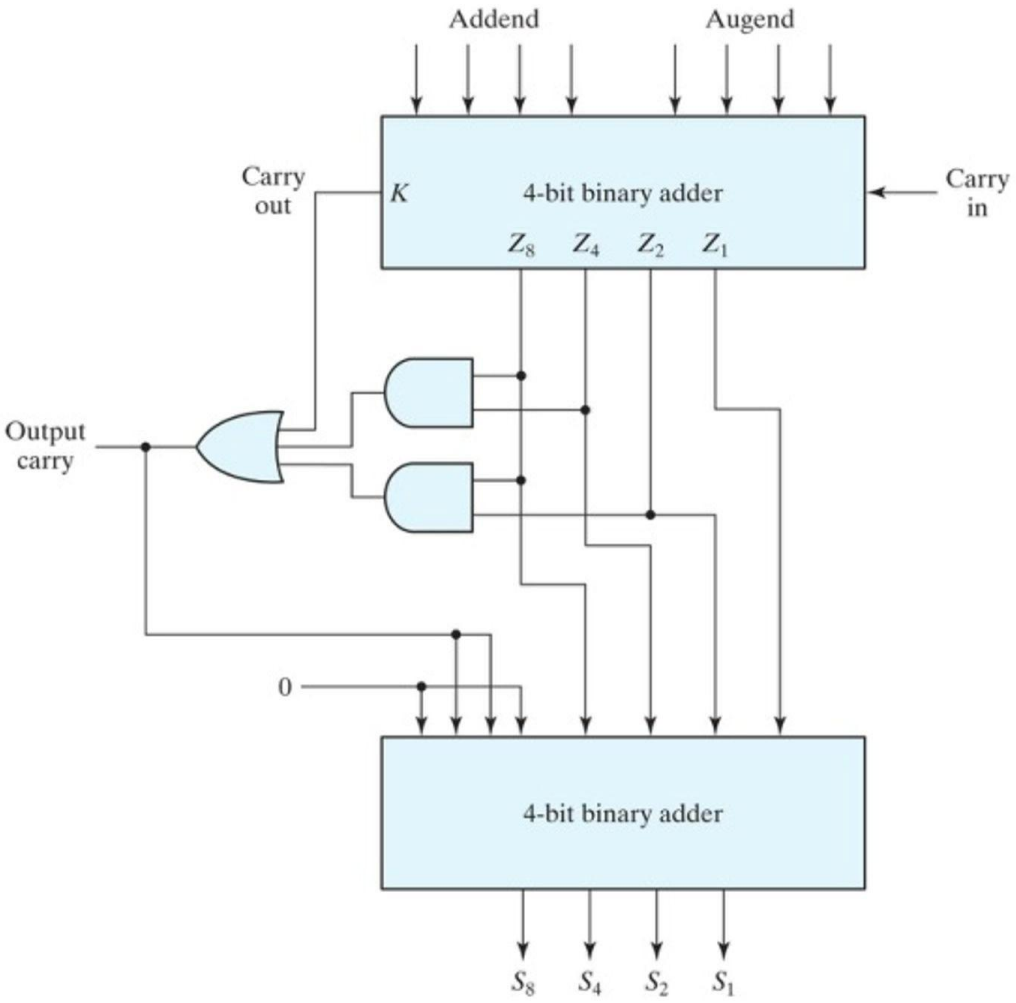
**Table 4.5**
*Derivation of BCD Adder*

| Binary Sum | | | | | BCD Sum | | | | | Decimal |
|---|---|---|---|---|---|---|---|---|---|---|
| $K$ | $Z_8$ | $Z_4$ | $Z_2$ | $Z_1$ | $C$ | $S_8$ | $S_4$ | $S_2$ | $S_1$ | |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 1 | 1 |
| 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 1 | 0 | 2 |
| 0 | 0 | 0 | 1 | 1 | 0 | 0 | 0 | 1 | 1 | 3 |
| 0 | 0 | 1 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 4 |
| 0 | 0 | 1 | 0 | 1 | 0 | 0 | 1 | 0 | 1 | 5 |
| 0 | 0 | 1 | 1 | 0 | 0 | 0 | 1 | 1 | 0 | 6 |
| 0 | 0 | 1 | 1 | 1 | 0 | 0 | 1 | 1 | 1 | 7 |
| 0 | 1 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 8 |
| 0 | 1 | 0 | 0 | 1 | 0 | 1 | 0 | 0 | 1 | 9 |
| 0 | 1 | 0 | 1 | 0 | 1 | 0 | 0 | 0 | 0 | 10 |
| 0 | 1 | 0 | 1 | 1 | 1 | 0 | 0 | 0 | 1 | 11 |
| 0 | 1 | 1 | 0 | 0 | 1 | 0 | 0 | 1 | 0 | 12 |
| 0 | 1 | 1 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 13 |
| 0 | 1 | 1 | 1 | 0 | 1 | 0 | 1 | 0 | 0 | 14 |
| 0 | 1 | 1 | 1 | 1 | 1 | 0 | 1 | 0 | 1 | 15 |
| 1 | 0 | 0 | 0 | 0 | 1 | 0 | 1 | 1 | 0 | 16 |
| 1 | 0 | 0 | 0 | 1 | 1 | 0 | 1 | 1 | 1 | 17 |
| 1 | 0 | 0 | 1 | 0 | 1 | 1 | 0 | 0 | 0 | 18 |
| 1 | 0 | 0 | 1 | 1 | 1 | 1 | 0 | 0 | 1 | 19 |

Copyright ©2012 Pearson Education, publishing as Prentice Hall


Copyright ©2013 by Pearson Education, Inc.
All rights reserved.

**FIGURE 4.14** Block diagram of a BCD adder

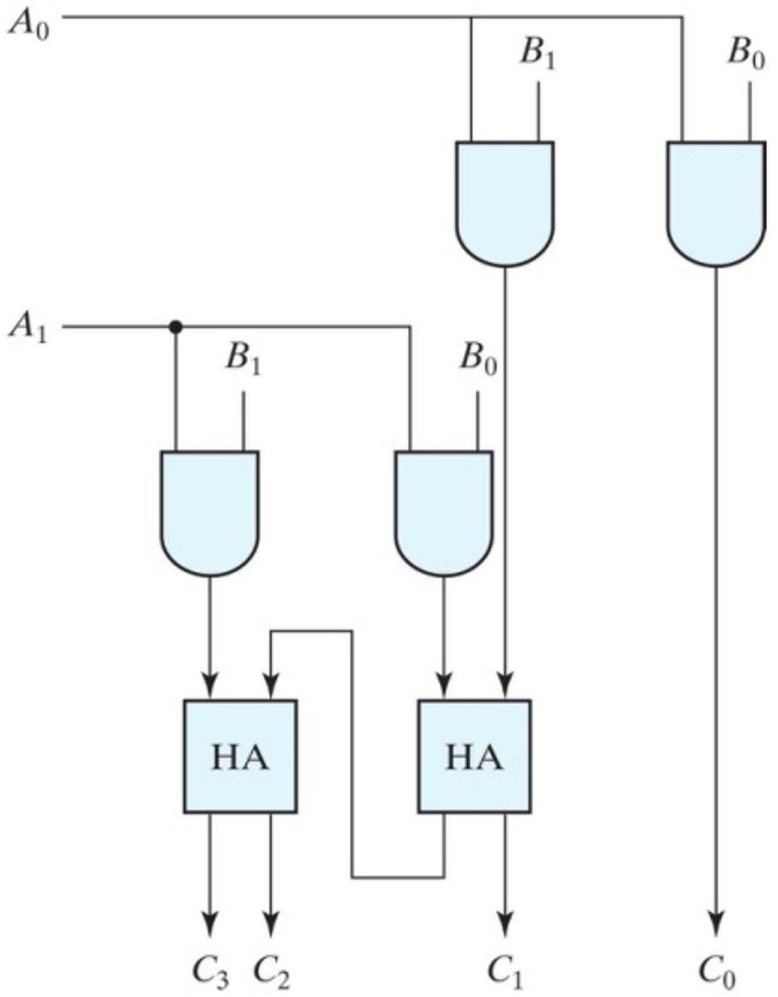



Copyright ©2013 Pearson Education, publishing as Prentice Hall


Copyright ©2013 by Pearson Education, Inc.
All rights reserved. PEARSON

**FIGURE 4.15** Two-bit by two-bit binary multiplier

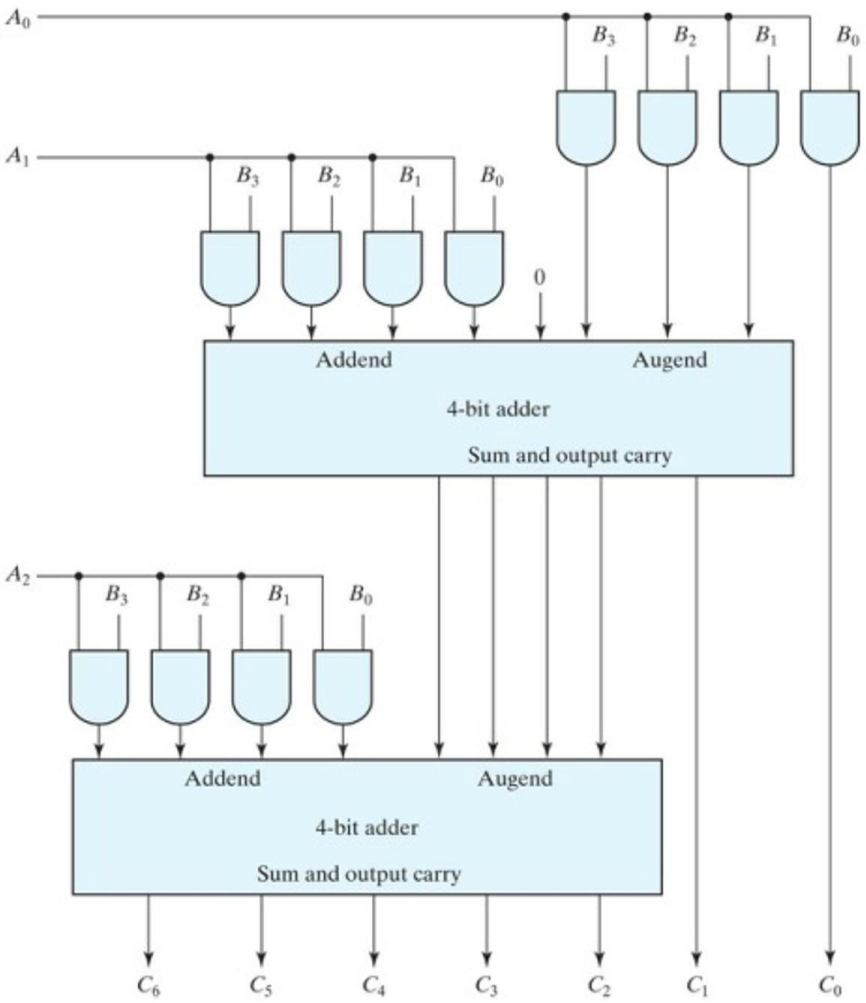
$$
\begin{array}{cccc}
 & & B_1 & B_0 \\
 & & A_1 & A_0 \\
\hline
 & & A_0B_1 & A_0B_0 \\
 & A_1B_1 & A_1B_0 & \\
\hline
C_3 & C_2 & C_1 & C_0
\end{array}
$$




Copyright ©2013 Pearson Education, publishing as Prentice Hall


Copyright ©2013 by Pearson Education, Inc.
All rights reserved. PEARSON

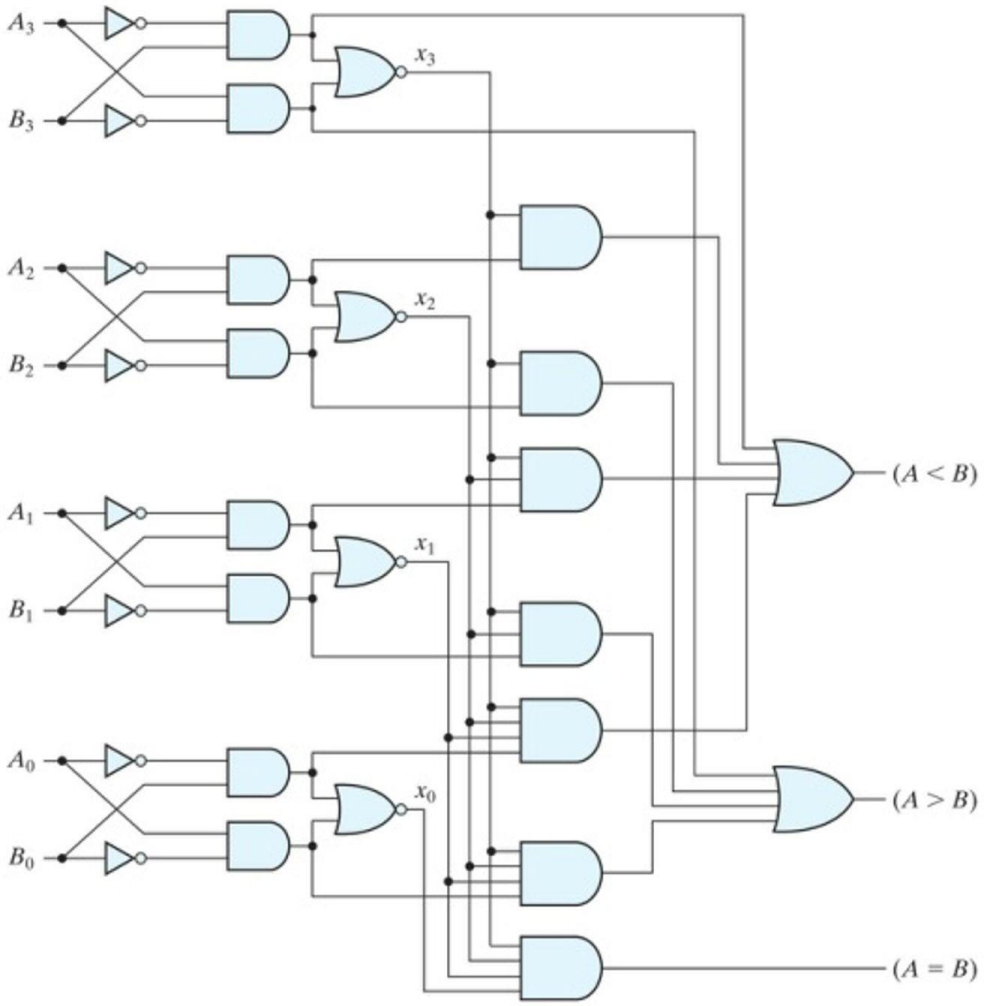
**FIGURE 4.16** Four-bit by three-bit binary multiplier




Copyright ©2013 Pearson Education, publishing as Prentice Hall


Copyright ©2013 by Pearson Education, Inc.
All rights reserved.
PEARSON

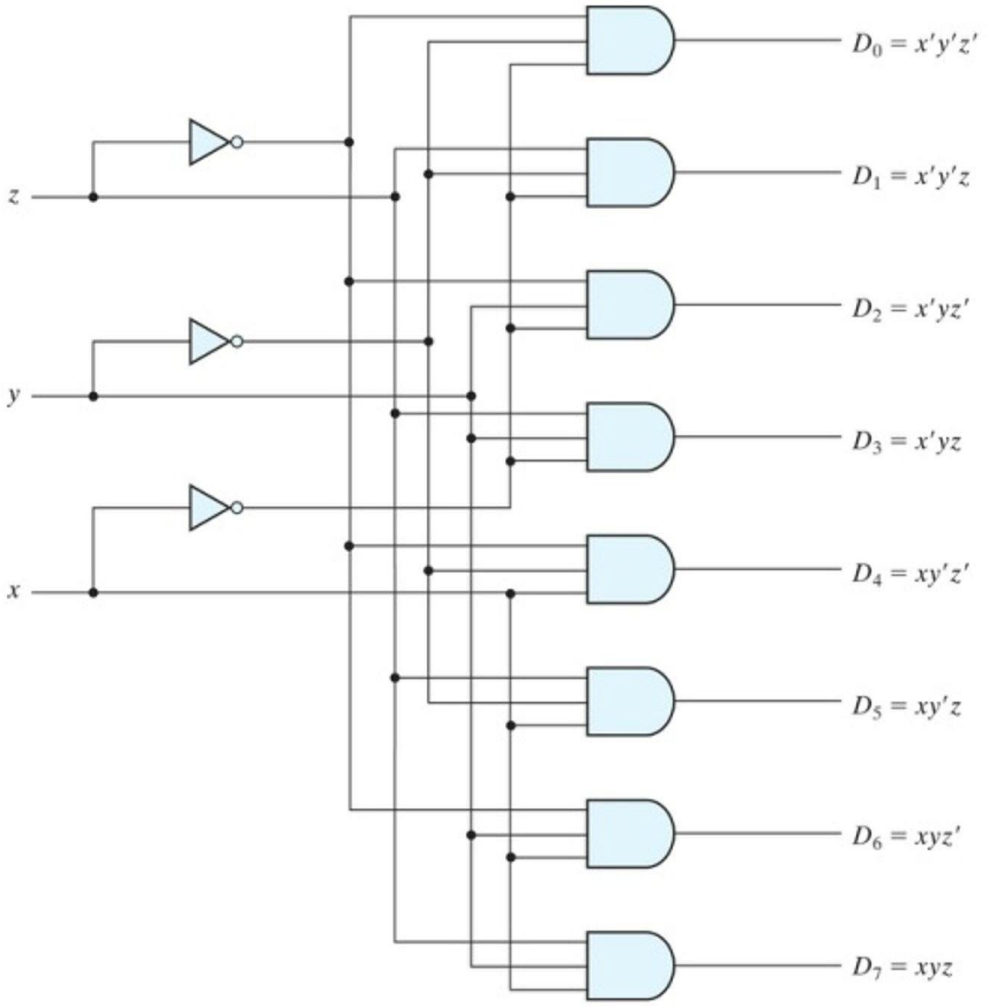
**FIGURE 4.17** Four-bit magnitude comparator




Copyright ©2013 Pearson Education, publishing as Prentice Hall


Copyright ©2013 by Pearson Education, Inc.
All rights reserved. PEARSON

**FIGURE 4.18** Three-to-eight-line decoder




Copyright ©2013 Pearson Education, publishing as Prentice Hall


Copyright ©2013 by Pearson Education, Inc.
All rights reserved. PEARSON

**Table 4.6** Truth Table of a Three-to-Eight-Line Decoder

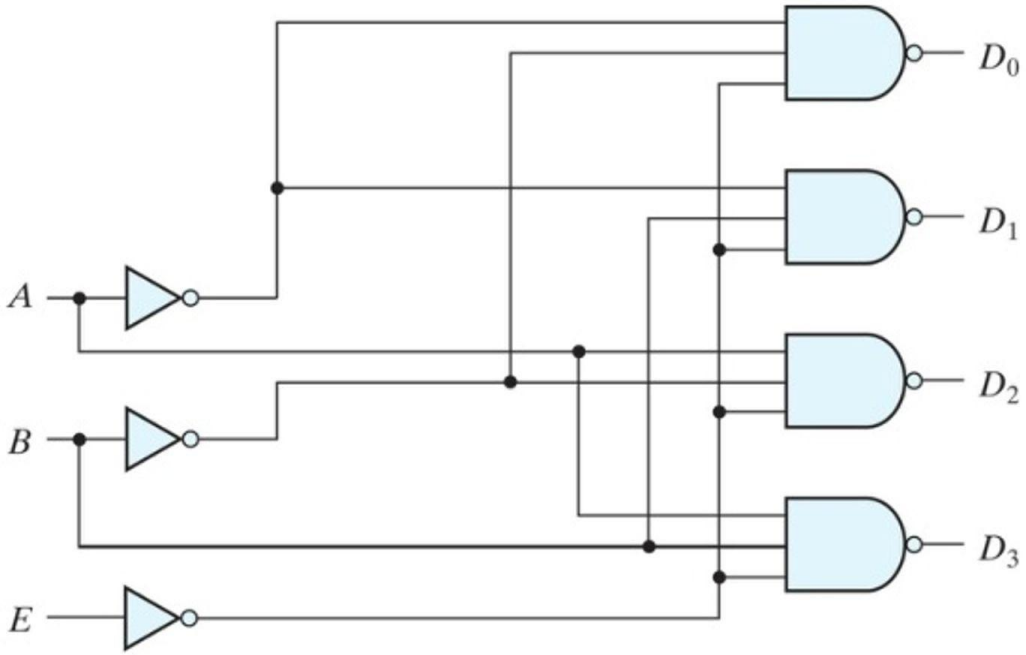
## Table 4.6
*Truth Table of a Three-to-Eight-Line Decoder*

| Inputs | | | Outputs | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| $x$ | $y$ | $z$ | $D_0$ | $D_1$ | $D_2$ | $D_3$ | $D_4$ | $D_5$ | $D_6$ | $D_7$ |
| 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 1 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 |
| 0 | 1 | 1 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 |
| 1 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 |
| 1 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 |
| 1 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 |
| 1 | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 |

Copyright ©2012 Pearson Education, publishing as Prentice Hall


Copyright ©2013 by Pearson Education, Inc.
All rights reserved. PEARSON

**FIGURE 4.19** Two-to-four-line decoder with enable input




(a) Logic diagram

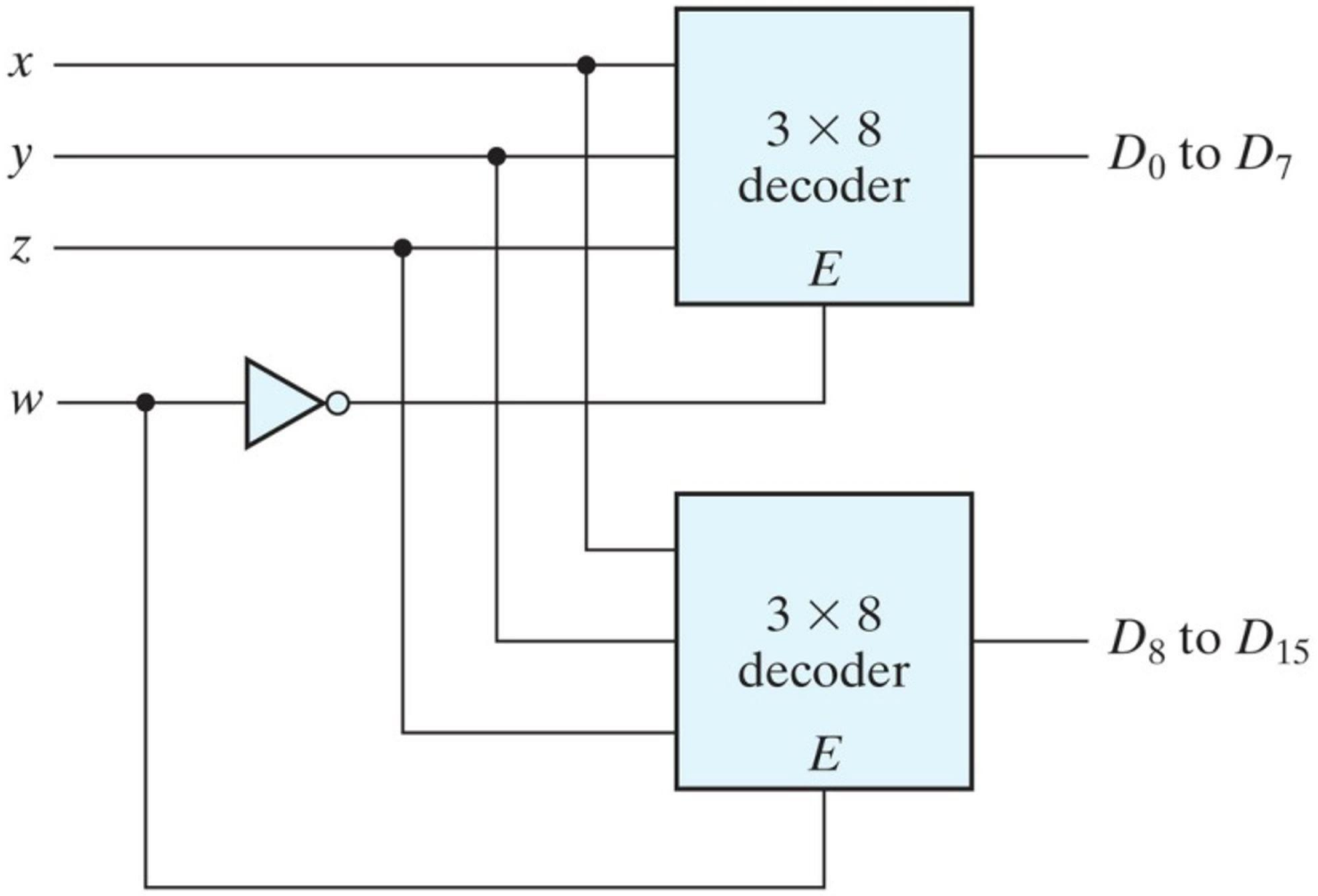
| $E$ | $A$ | $B$ | $D_0$ | $D_1$ | $D_2$ | $D_3$ |
|---|---|---|---|---|---|---|
| 1 | $X$ | $X$ | 1 | 1 | 1 | 1 |
| 0 | 0 | 0 | 0 | 1 | 1 | 1 |
| 0 | 0 | 1 | 1 | 0 | 1 | 1 |
| 0 | 1 | 0 | 1 | 1 | 0 | 1 |
| 0 | 1 | 1 | 1 | 1 | 1 | 1 |

(b) Truth table

Copyright ©2013 Pearson Education, publishing as Prentice Hall


Copyright ©2013 by Pearson Education, Inc.
All rights reserved.

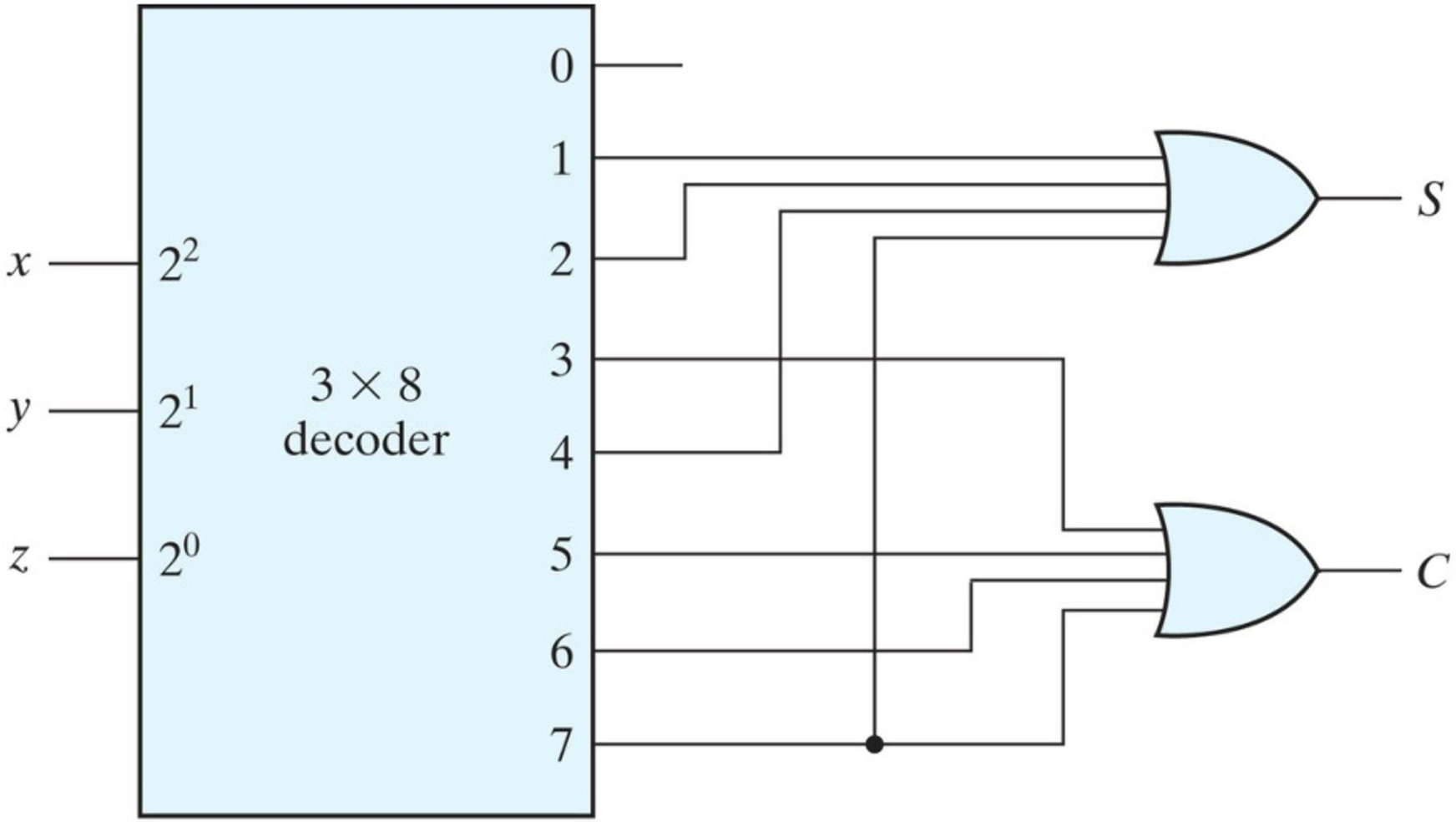
**FIGURE 4.20** 4 × 16 decoder constructed with two 3 × 8 decoders




Copyright ©2013 Pearson Education, publishing as Prentice Hall


Copyright ©2013 by Pearson Education, Inc.
All rights reserved.
PEARSON

**FIGURE 4.21** Implementation of a full adder with a decoder




Copyright ©2013 Pearson Education, publishing as Prentice Hall


Copyright ©2013 by Pearson Education, Inc.
All rights reserved.
PEARSON

**Table 4.7** Truth Table of an Octal-to-Binary Encoder

## Table 4.7
*Truth Table of an Octal-to-Binary Encoder*

| Inputs | | | | | | | | Outputs | | |
|---|---|---|---|---|---|---|---|---|---|---|
| $D_0$ | $D_1$ | $D_2$ | $D_3$ | $D_4$ | $D_5$ | $D_6$ | $D_7$ | $x$ | $y$ | $z$ |
| 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 |
| 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 |
| 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 1 | 1 |
| 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 1 | 0 | 1 |
| 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 1 | 1 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 |

Copyright ©2012 Pearson Education, publishing as Prentice Hall


Copyright ©2013 by Pearson Education, Inc.
All rights reserved. PEARSON

**Table 4.8** Truth Table of a Priority Encoder

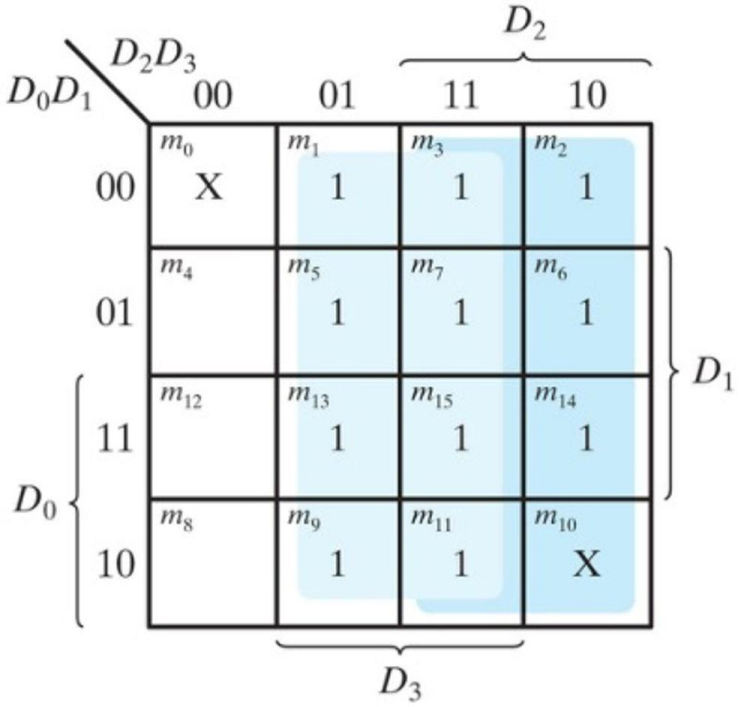
# Table 4.8
## *Truth Table of a Priority Encoder*

| Inputs | | | | Outputs | | |
|---|---|---|---|---|---|---|
| $D_0$ | $D_1$ | $D_2$ | $D_3$ | $x$ | $y$ | $V$ |
| 0 | 0 | 0 | 0 | X | X | 0 |
| 1 | 0 | 0 | 0 | 0 | 0 | 1 |
| X | 1 | 0 | 0 | 0 | 1 | 1 |
| X | X | 1 | 0 | 1 | 0 | 1 |
| X | X | X | 1 | 1 | 1 | 1 |

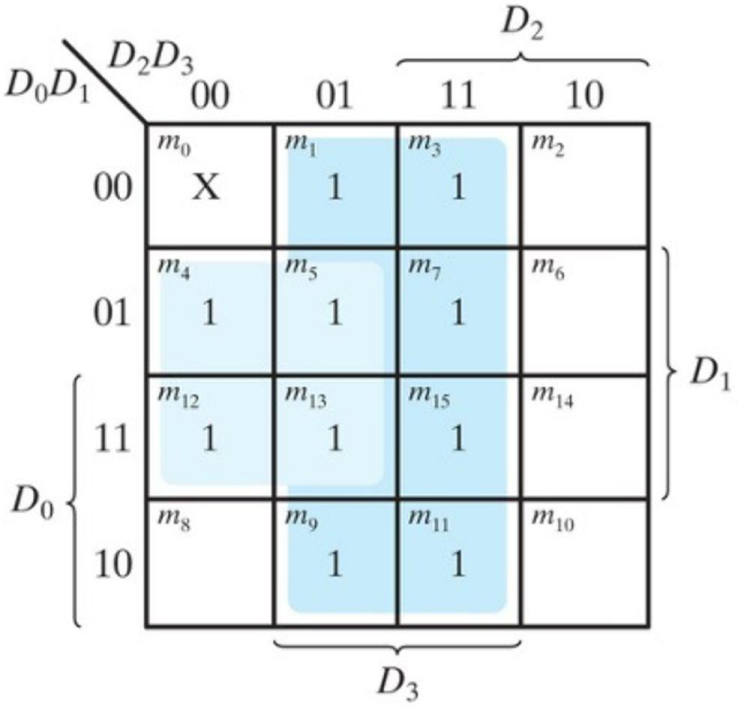
Copyright ©2012 Pearson Education, publishing as Prentice Hall


Copyright ©2013 by Pearson Education, Inc.
All rights reserved.

**FIGURE 4.22** Maps for a priority encoder




$x = D_2 + D_3$

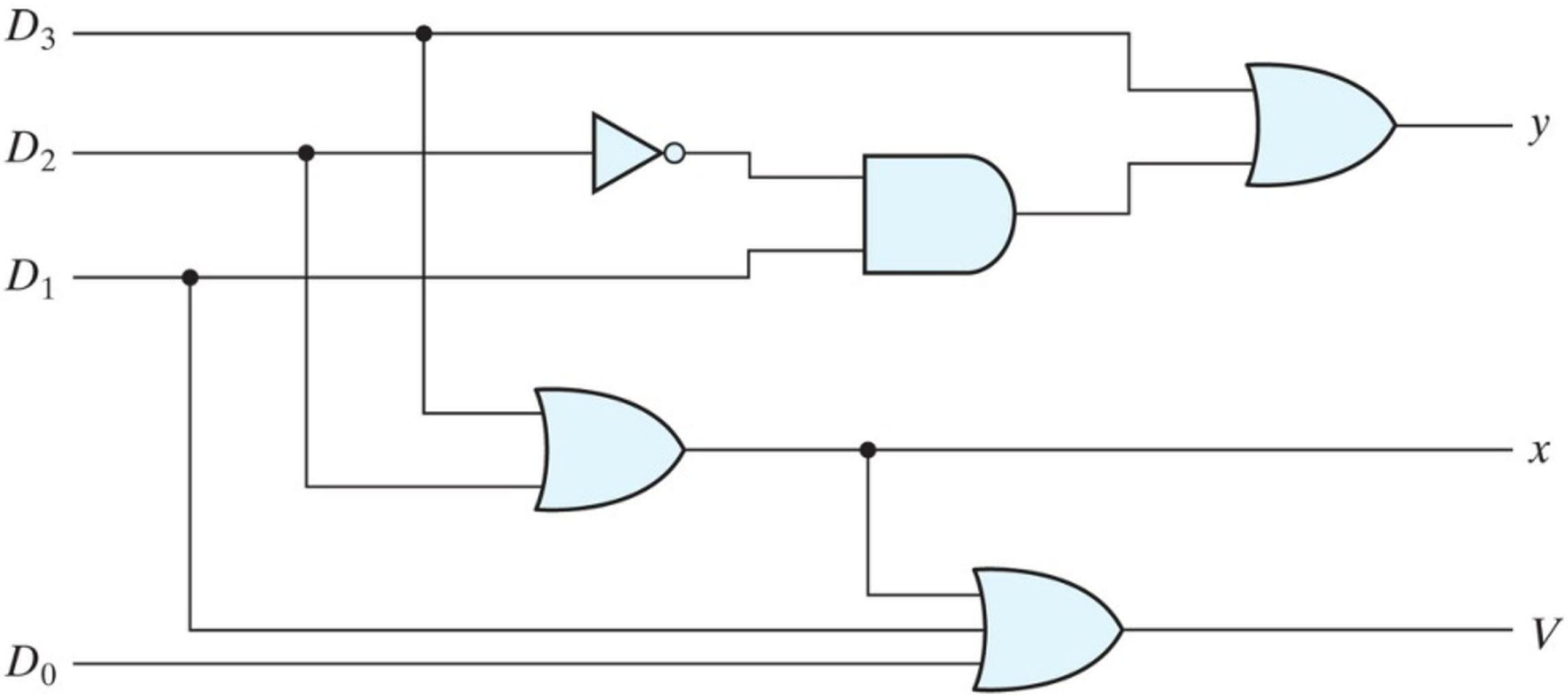



$y = D_3 + D_1D'_2$

Copyright ©2013 Pearson Education, publishing as Prentice Hall


Copyright ©2013 by Pearson Education, Inc.
All rights reserved. PEARSON

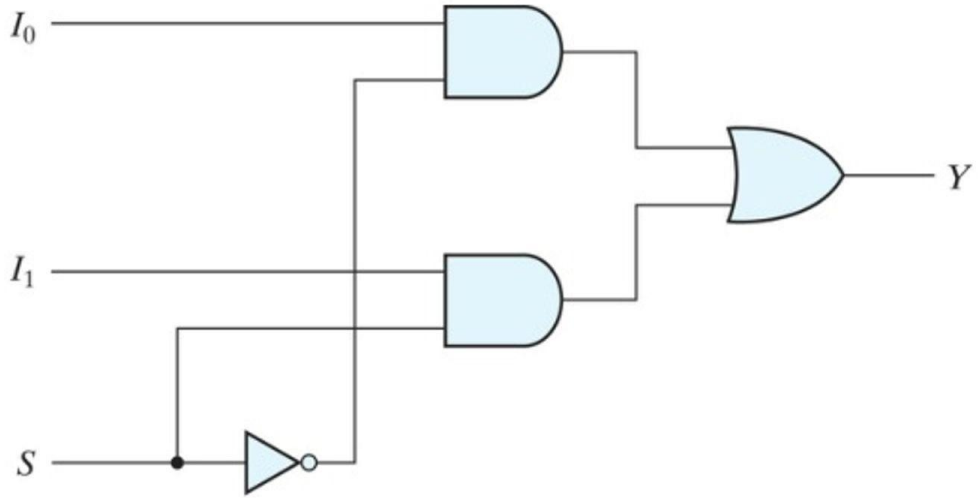
**FIGURE 4.23** Four-input priority encoder



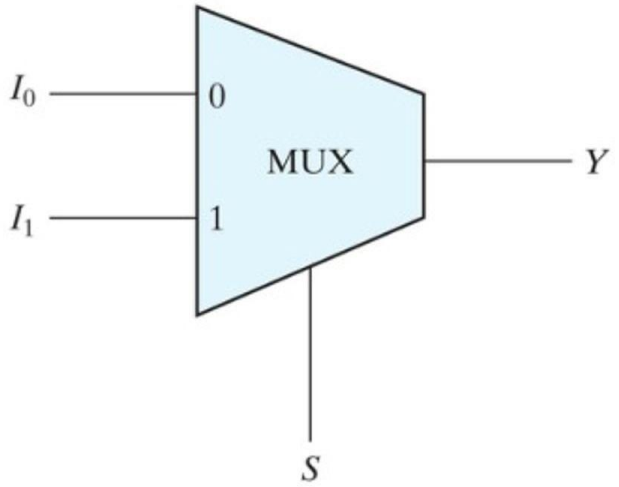

Copyright ©2013 Pearson Education, publishing as Prentice Hall


Copyright ©2013 by Pearson Education, Inc.
All rights reserved. PEARSON

**FIGURE 4.24** Two-to-one-line multiplexer




(a) Logic diagram

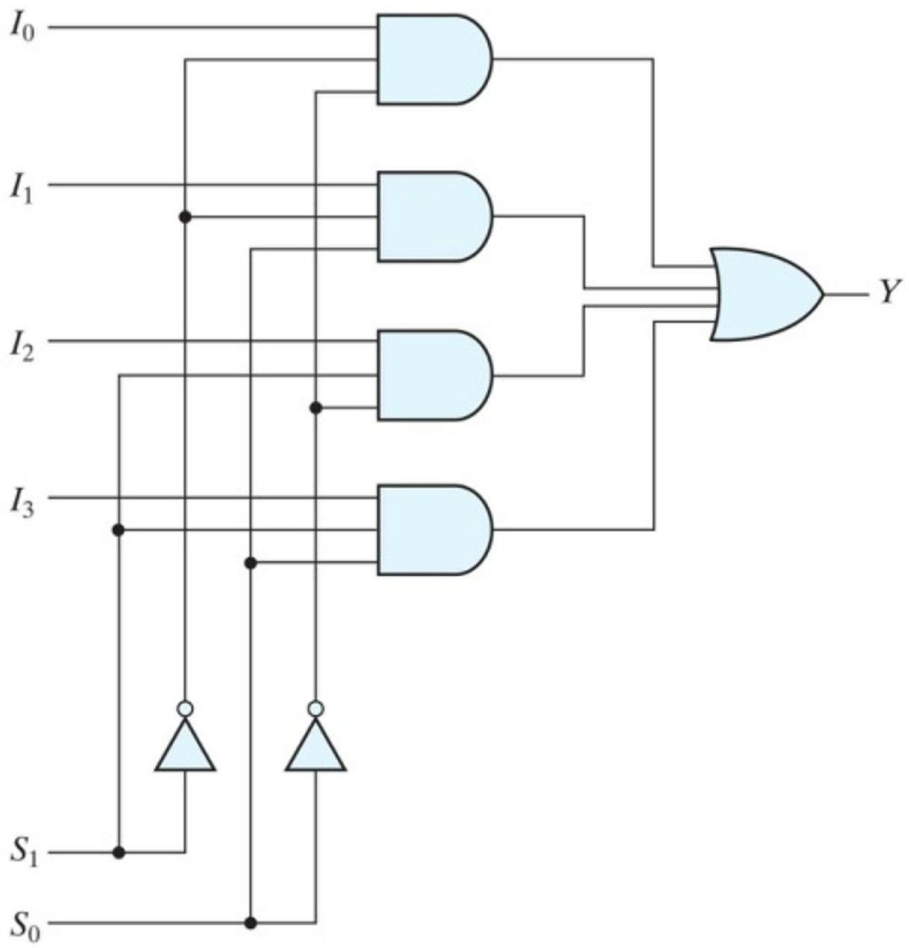



(b) Block diagram

Copyright ©2013 Pearson Education, publishing as Prentice Hall


Copyright ©2013 by Pearson Education, Inc.
All rights reserved. PEARSON

**FIGURE 4.25** Four-to-one-line multiplexer




(a) Logic diagram

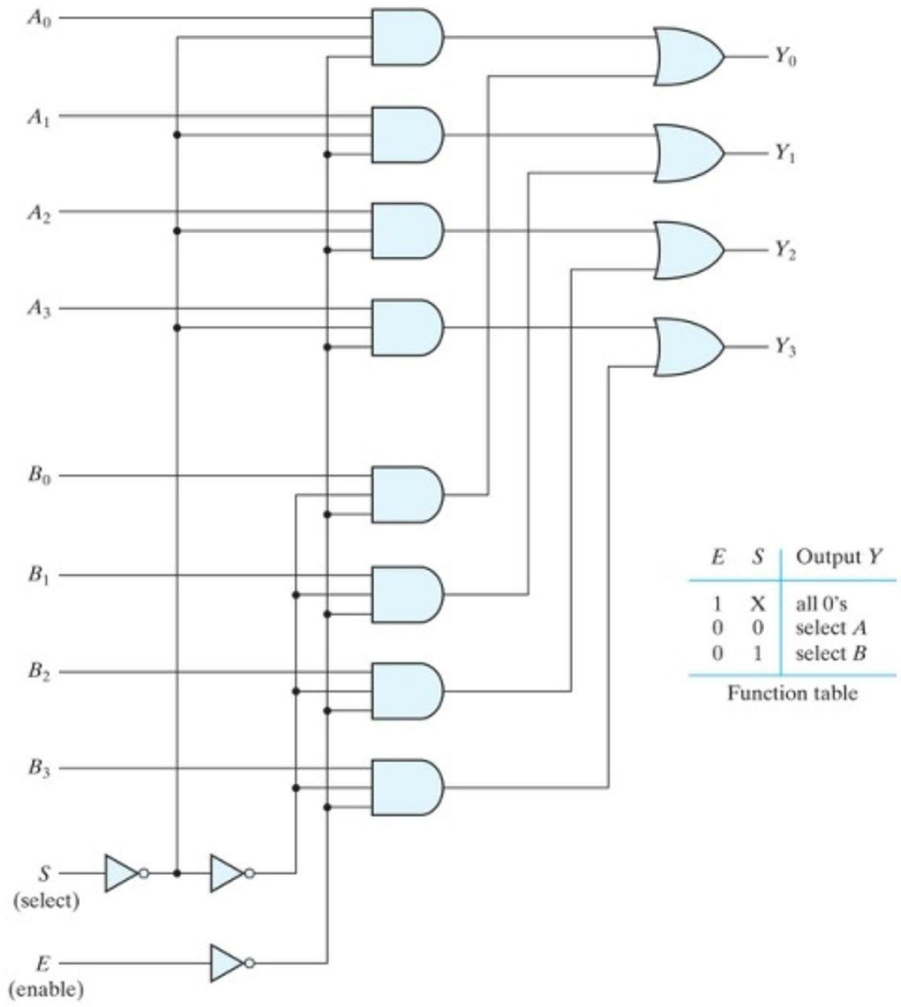
| $S_1$ | $S_0$ | $Y$ |
|---|---|---|
| 0 | 0 | $I_0$ |
| 0 | 1 | $I_1$ |
| 1 | 0 | $I_2$ |
| 1 | 1 | $I_3$ |

(b) Function table

Copyright ©2013 Pearson Education, publishing as Prentice Hall


Copyright ©2013 by Pearson Education, Inc.
All rights reserved.

**FIGURE 4.26** Quadruple two-to-one-line multiplexer




| $E$ | $S$ | Output $Y$ |
|---|---|---|
| 1 | X | all 0's |
| 0 | 0 | select $A$ |
| 0 | 1 | select $B$ |

Function table

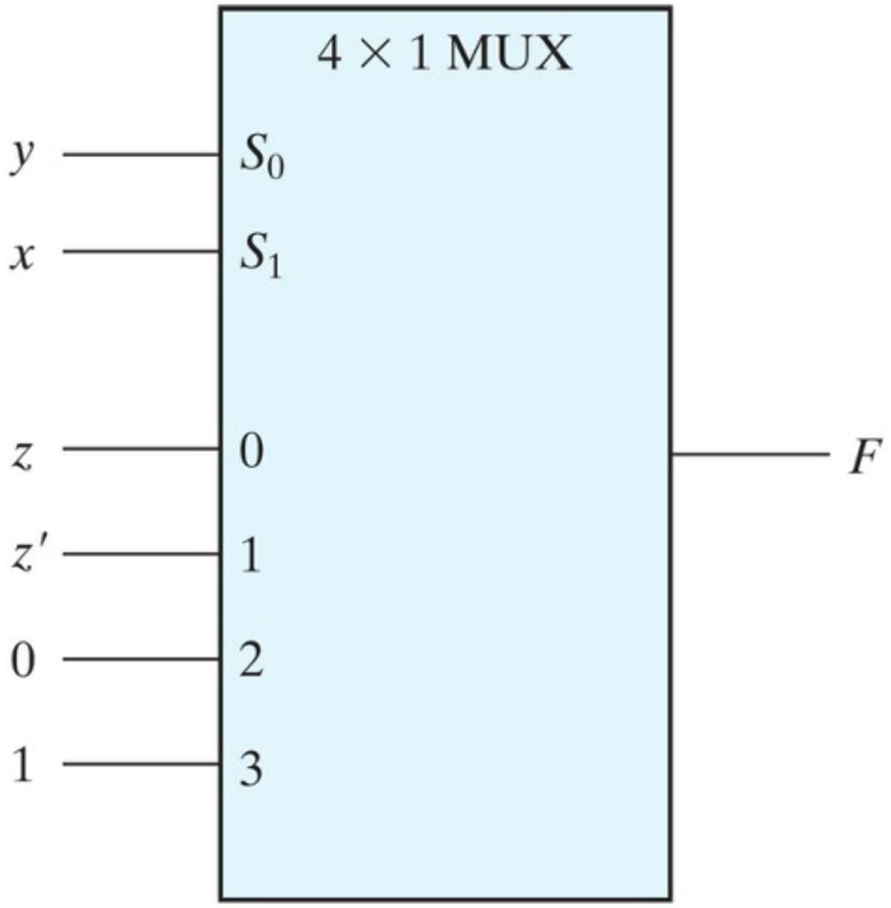
Copyright ©2013 Pearson Education, publishing as Prentice Hall


Copyright ©2013 by Pearson Education, Inc.
All rights reserved.
PEARSON

**FIGURE 4.27** Implementing a Boolean function with a multiplexer

| $x$ | $y$ | $z$ | $F$ | |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | $F = z$ |
| 0 | 0 | 1 | 1 | |
| 0 | 1 | 0 | 1 | $F = z'$ |
| 0 | 1 | 1 | 0 | |
| 1 | 0 | 0 | 0 | $F = 0$ |
| 1 | 0 | 1 | 0 | |
| 1 | 1 | 0 | 1 | $F = 1$ |
| 1 | 1 | 1 | 1 | |

(a) Truth table



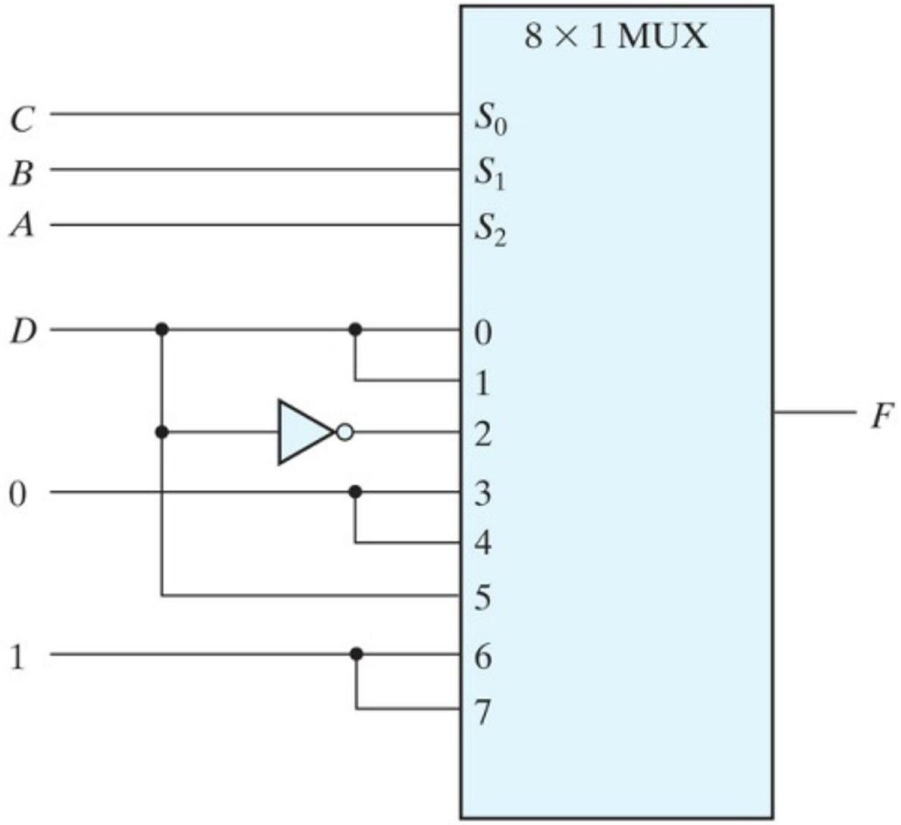

(b) Multiplexer implementation

Copyright ©2013 Pearson Education, publishing as Prentice Hall


Copyright ©2013 by Pearson Education, Inc.
All rights reserved.
PEARSON

**FIGURE 4.28** Implementing a four-input function with a multiplexer

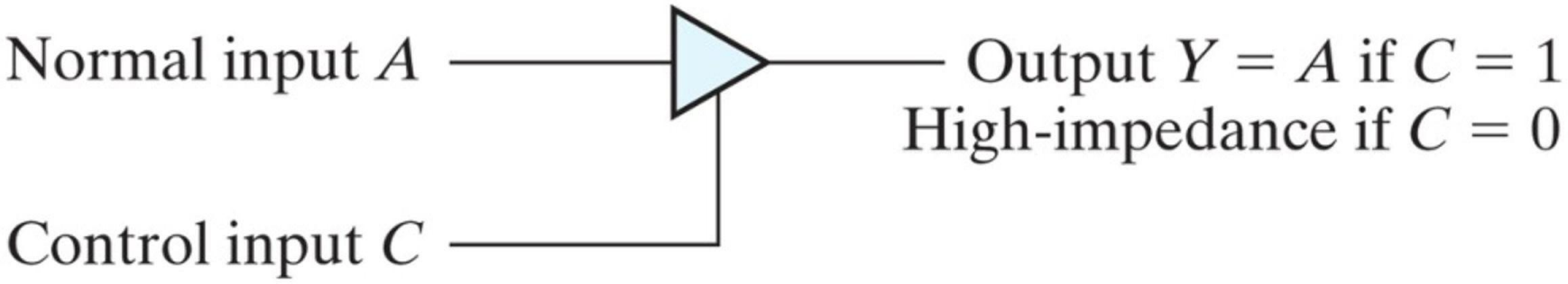
| $A$ | $B$ | $C$ | $D$ | $F$ | |
|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | $F = D$ |
| 0 | 0 | 0 | 1 | 1 | |
| 0 | 0 | 1 | 0 | 0 | $F = D$ |
| 0 | 0 | 1 | 1 | 1 | |
| 0 | 1 | 0 | 0 | 1 | $F = D'$ |
| 0 | 1 | 0 | 1 | 0 | |
| 0 | 1 | 1 | 0 | 0 | $F = 0$ |
| 0 | 1 | 1 | 1 | 0 | |
| 1 | 0 | 0 | 0 | 0 | $F = 0$ |
| 1 | 0 | 0 | 1 | 0 | |
| 1 | 0 | 1 | 0 | 0 | $F = D$ |
| 1 | 0 | 1 | 1 | 1 | |
| 1 | 1 | 0 | 0 | 1 | $F = 1$ |
| 1 | 1 | 0 | 1 | 1 | |
| 1 | 1 | 1 | 0 | 1 | $F = 1$ |
| 1 | 1 | 1 | 1 | 1 | |




Copyright ©2013 Pearson Education, publishing as Prentice Hall


Copyright ©2013 by Pearson Education, Inc.
All rights reserved. PEARSON

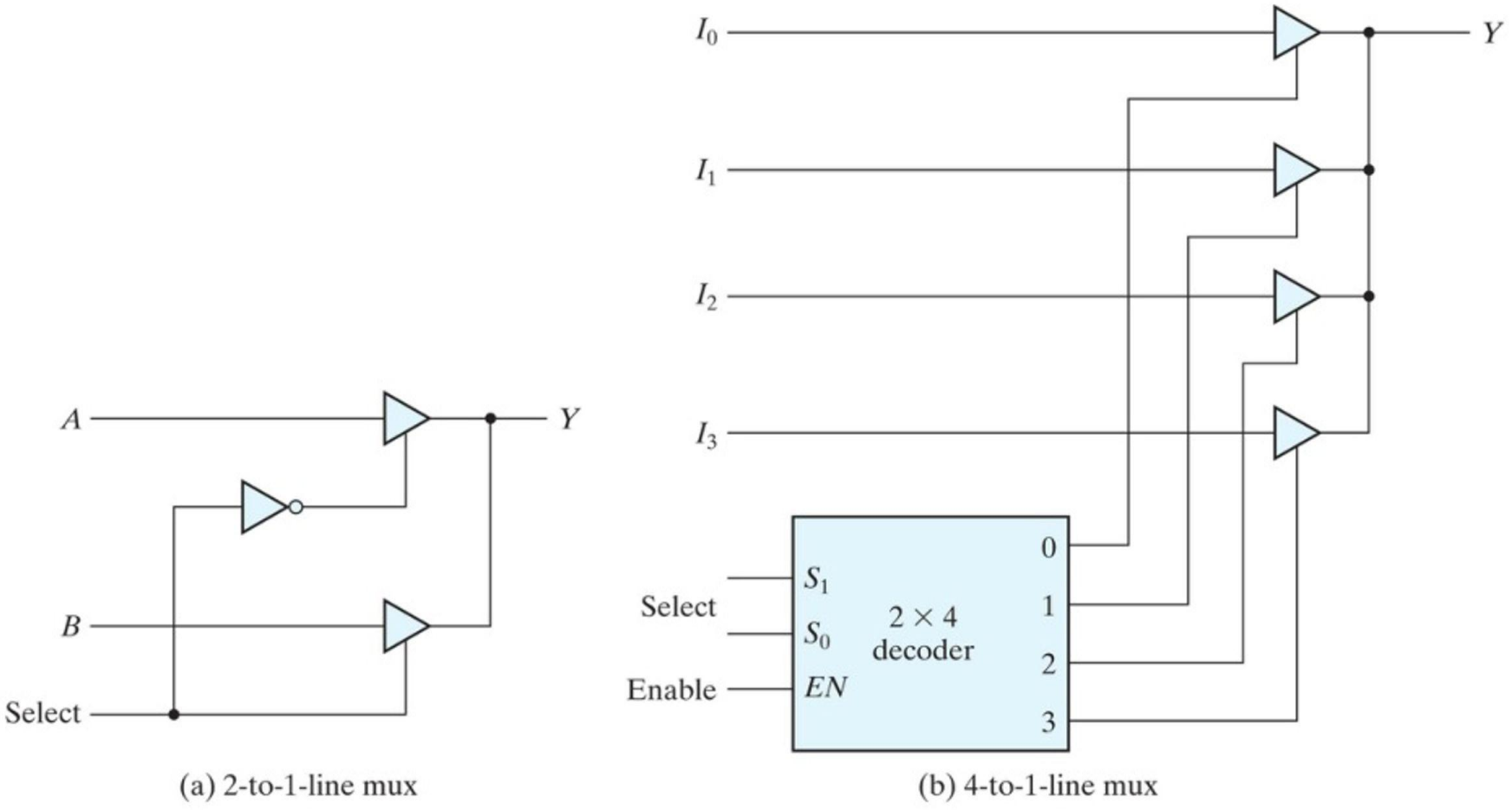
**FIGURE 4.29** Graphic symbol for a three-state buffer




Copyright ©2013 Pearson Education, publishing as Prentice Hall


Copyright ©2013 by Pearson Education, Inc.
All rights reserved. PEARSON

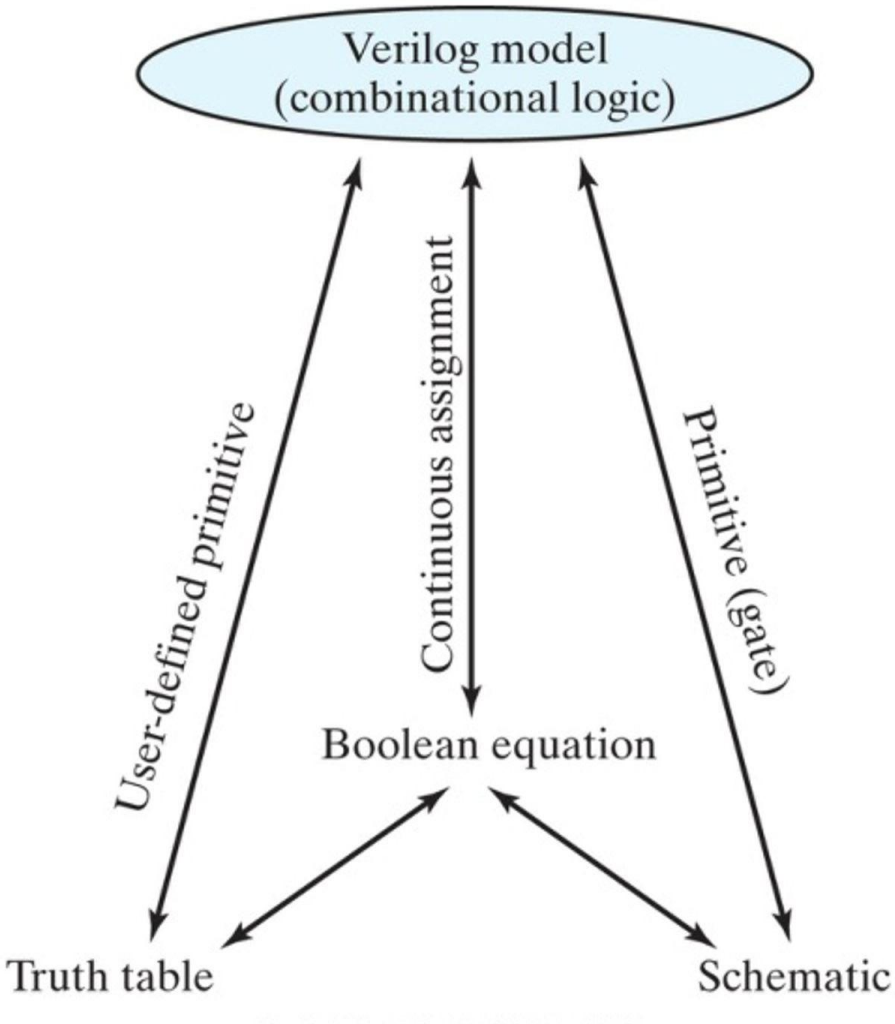
**FIGURE 4.30** Multiplexers with three-state gates




Copyright ©2013 Pearson Education, publishing as Prentice Hall


Copyright ©2013 by Pearson Education, Inc.
All rights reserved.
PEARSON

**FIGURE 4.31** Relationship of Verilog constructs to truth tables, Boolean equations, and schematics




Copyright ©2013 Pearson Education, publishing as Prentice Hall


Copyright ©2013 by Pearson Education, Inc.
All rights reserved. PEARSON

**Table 4.9** Truth Table for Predefined Primitive Gates

## Table 4.9
### *Truth Table for Predefined Primitive Gates*

| and | 0 | 1 | x | z |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 1 | 0 | 1 | x | x |
| x | 0 | x | x | x |
| z | 0 | x | x | x |

| or | 0 | 1 | x | z |
|---|---|---|---|---|
| 0 | 0 | 1 | x | x |
| 1 | 1 | 1 | 1 | 1 |
| x | x | 1 | x | x |
| z | x | 1 | x | x |

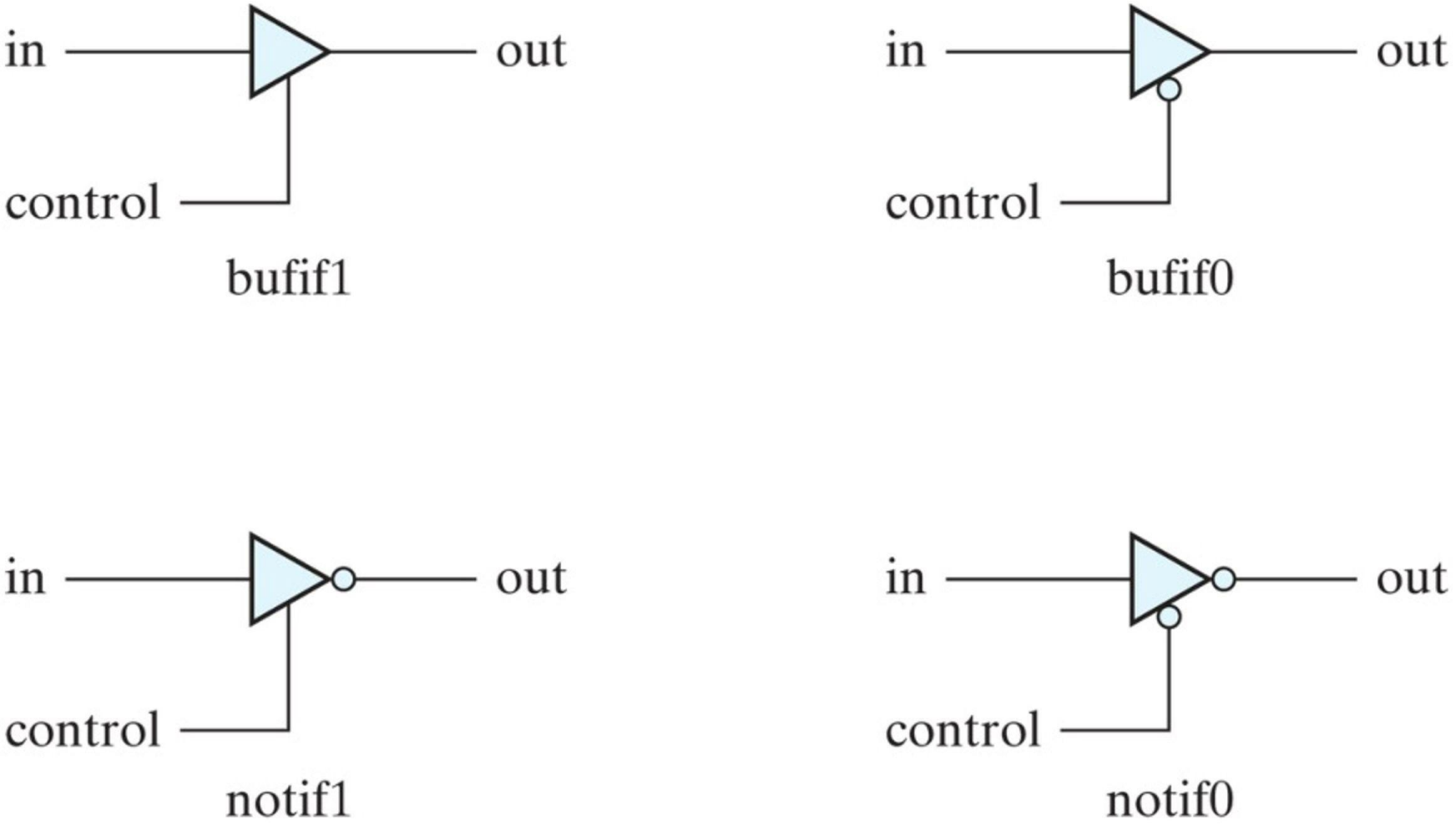
| xor | 0 | 1 | x | z |
|---|---|---|---|---|
| 0 | 0 | 1 | x | x |
| 1 | 1 | 0 | x | x |
| x | x | x | x | x |
| z | x | x | x | x |

| not | input | output |
|---|---|---|
| | 0 | 1 |
| | 1 | 0 |
| | x | x |
| | z | x |

Copyright ©2012 Pearson Education, publishing as Prentice Hall


Copyright ©2013 by Pearson Education, Inc.
All rights reserved.

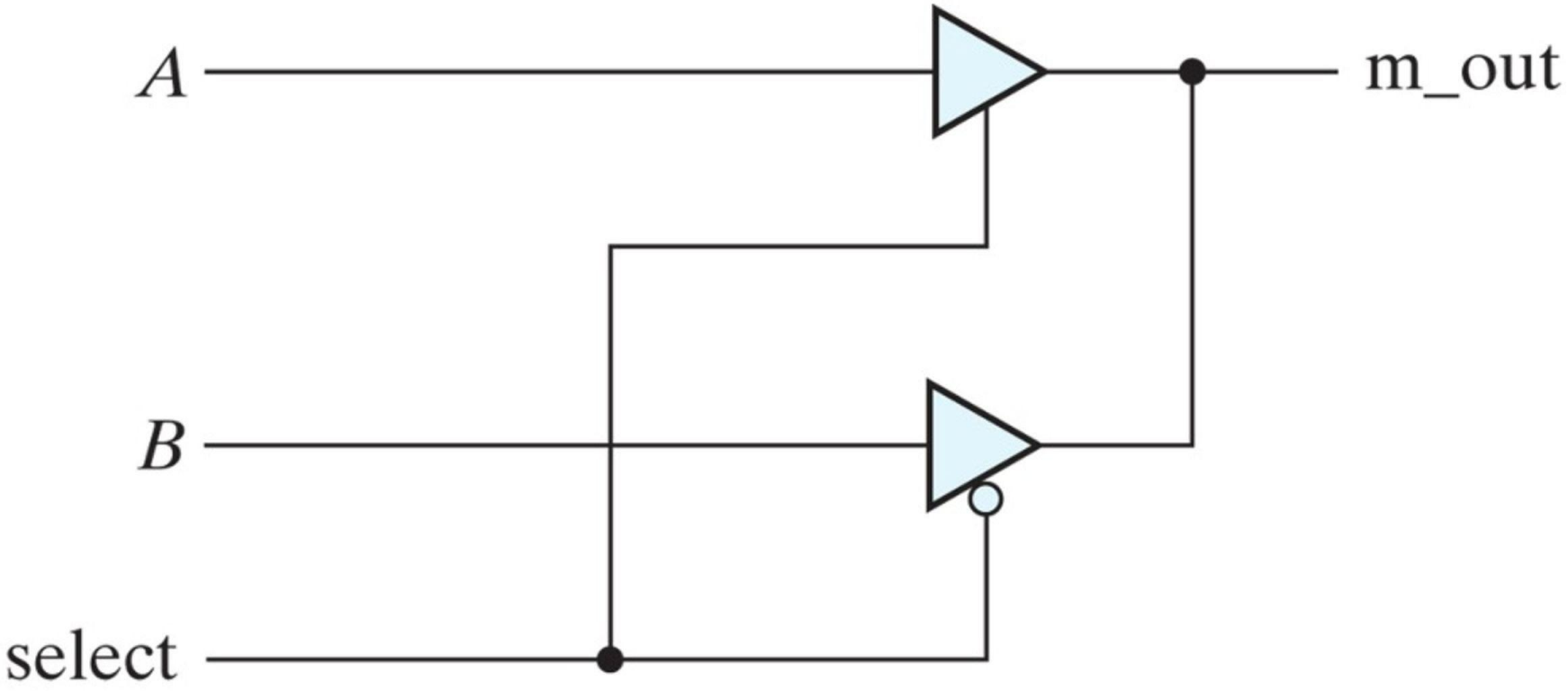
**FIGURE 4.32** Three-state gates




Copyright ©2013 Pearson Education, publishing as Prentice Hall


Copyright ©2013 by Pearson Education, Inc.
All rights reserved.
PEARSON

**FIGURE 4.33** Two-to-one-line multiplexer with three-state buffers




Copyright ©2013 Pearson Education, publishing as Prentice Hall


Copyright ©2013 by Pearson Education, Inc.
All rights reserved. PEARSON

**Table 4.10** Some Verilog HDL Operators

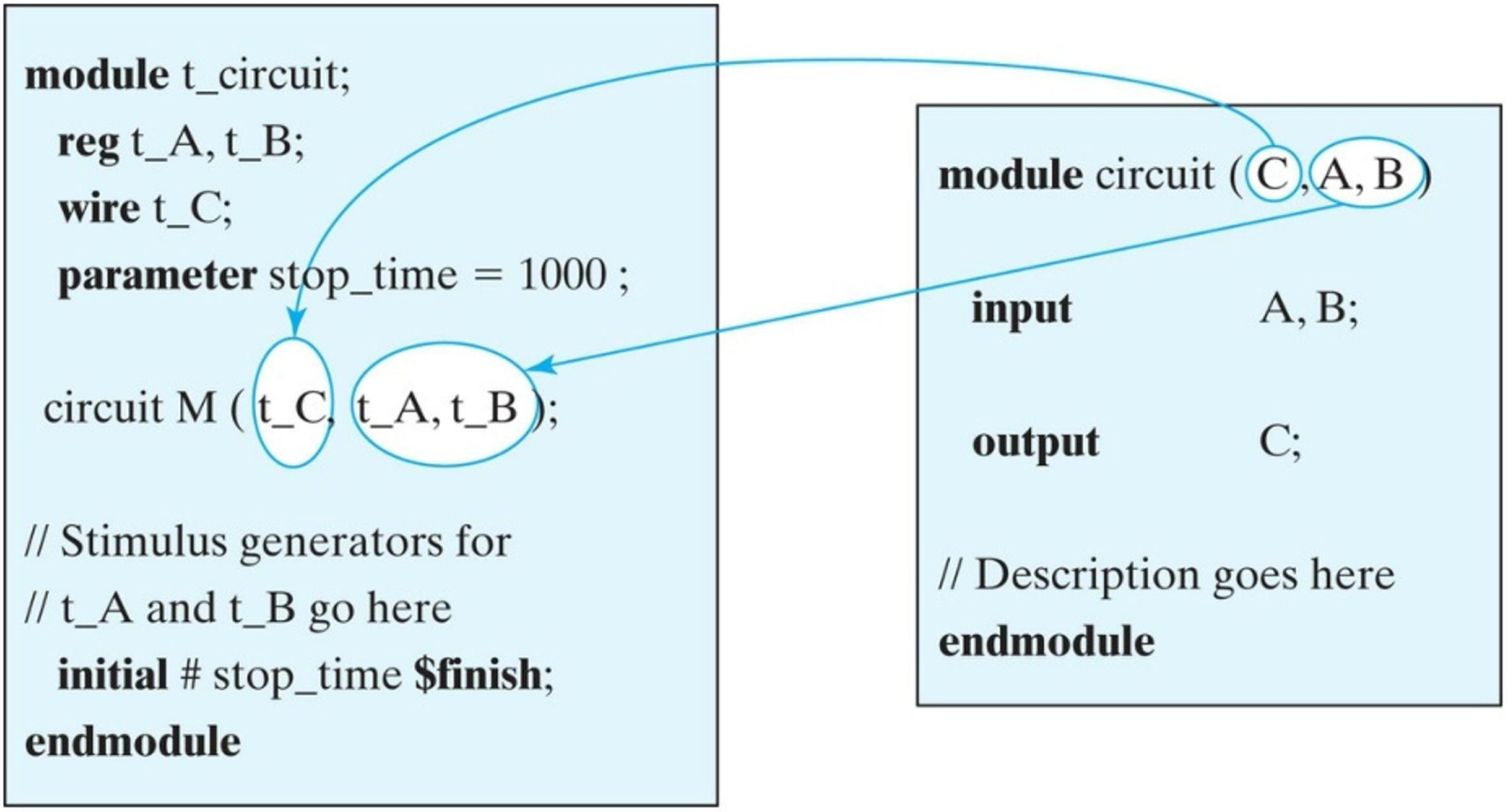
## Table 4.10
*Some Verilog HDL Operators*

| Symbol | Operation | Symbol | Operation |
|---|---|---|---|
| + | binary addition | | |
| − | binary subtraction | | |
| & | bitwise AND | && | logical AND |
| \| | bitwise OR | \|\| | logical OR |
| ^ | bitwise XOR | | |
| ~ | bitwise NOT | ! | logical NOT |
| = = | equality | | |
| > | greater than | | |
| < | less than | | |
| {} | concatenation | | |
| ?: | conditional | | |

Copyright ©2012 Pearson Education, publishing as Prentice Hall


Copyright ©2013 by Pearson Education, Inc.
All rights reserved.

**FIGURE 4.34** Interaction between stimulus and design modules




Copyright ©2013 Pearson Education, publishing as Prentice Hall


Copyright ©2013 by Pearson Education, Inc.
All rights reserved. PEARSON